
MODELAGEM DE PROGRAMAS PARALELOS

Computação Paralela: Unidade 02

Agosto, 2026

POSICIONAMENTO NA DISCIPLINA

UNIDADE 01 CONCLUÍDA

Que máquinas existem e como medir se estamos aproveitando-as.

Arquiteturas, classificações, speed-up, eficiência, Amdahl e Gustafson.

UNIDADE 02 EM CURSO

Como projetar o programa paralelo e escolher a ferramenta adequada.

Decomposição, comunicação, granularidade, mapeamento, modelos e interfaces.

UNIDADES 03 E 04

Como implementar, em laboratório.

OpenMP para memória compartilhada, MPI para troca de mensagens.

ROTEIRO DA UNIDADE

- 1 | Onde paralelizar: medir antes de decidir
- 2 | **PCAM**: particionar, comunicar, aglomerar, mapear
- 3 | Os cinco modelos de aplicação
- 4 | Estudo de caso: um problema, cinco modelagens
- 5 | Modelos de programação e interfaces: OpenMP e MPI

A P1 versa sobre as Unidades 01 e 02. Este é o último conteúdo conceitual antes da prova.

ANTES DE PARALELIZAR

Dificuldades inerentes e identificação de oportunidades

DIFICULDADES INERENTES À PROGRAMAÇÃO PARALELA

PARTICIONAMENTO

Definir como dividir **dados e código**, sem que exista uma resposta única.

SINCRONIZAÇÃO

Dependências de operação e **condições de corrida** que não existem no sequencial.

BALANCEAMENTO

Um processo ocioso é recurso alocado e não utilizado; o tempo total é o do processo **mais lento**.

Somam-se a essas: deadlocks de comunicação, resultados não determinísticos entre execuções e defeitos que só se manifestam em larga escala.

Nenhuma dessas dificuldades é eliminada por uma ferramenta melhor: todas se resolvem **na modelagem**.

AS TRÊS QUESTÕES CENTRAIS DA MODELAGEM

As decisões de modelagem que mais afetam o desempenho se concentram em três questões:

- 1 | Quantos processadores usar?
- 2 | Como distribuir a carga entre os envolvidos?
- 3 | Quando comunicar e sincronizar?

As três afetam **diretamente** o desempenho final, e todas são decisões **de projeto**, não de codificação.

PROGRAMAÇÃO VS. ARQUITETURA

O ideal seria programar sem conhecer a máquina. Em computação paralela isso **não é possível**: o conhecimento da arquitetura influencia diretamente o desempenho.

A máquina alvo determina três decisões de modelagem:

O PARADIGMA

Memória compartilhada ou troca de mensagens? Há necessidade de comunicação explícita?

A FERRAMENTA

OpenMP, MPI, CUDA: cada uma expressa o paralelismo de forma distinta.

A MODELAGEM

A granularidade viável e o mapeamento dependem da latência e da banda da máquina.

IDENTIFICAÇÃO DOS TRECHOS A PARALELIZAR

Um procedimento em quatro passos, aplicado ao programa sequencial:

- 1** | **Medir** o tempo de execução por trecho, com uma entrada realista.
- 2** | **Ordenar** os trechos pela fração do tempo total que cada um consome.
- 3** | Nos trechos mais custosos, localizar o **trabalho repetido**: a mesma operação aplicada a muitos elementos ou a muitos casos.
- 4** | Verificar a **ordem**: as repetições podem ocorrer em qualquer sequência, ou cada uma precisa do resultado da anterior?

Os passos 1 e 2 são **medição**; os passos 3 e 4 são **leitura do código**. Podemos usar ferramentas de **profiling** para ajudar no processo.

OBTENÇÃO DO PERFIL DE EXECUÇÃO

Três abordagens para a mesma pergunta, com custos e alcances distintos:

INSTRUMENTAÇÃO MANUAL

```
t0 = get_time_seconds();
filtro_gaussiano(...);
t_gauss = get_time_seconds() - t0;
```

O programador escolhe as fases. É a única que separa o programa em partes com significado de projeto e que fornece diretamente s e p .

GPROF

```
gcc -O3 -pg -fno-inline prog.c
./prog          # gera gmon.out
gprof -b -p prog gmon.out
```

Instrumenta cada função em tempo de compilação. Fornece o tempo por função e a **contagem exata de chamadas**.

PERF

```
perf record -F 999 ./prog
perf report --stdio \
  --no-children -g none
```

Interrompe o processo 999 vezes por segundo e registra a posição de execução. Dispensa recompilação e **alcança as bibliotecas**.

A perfilagem exige **entrada representativa**. Um caso de teste reduzido costuma apresentar perfil completamente diferente do caso de produção.

INSTRUMENTAÇÃO MANUAL: RELATÓRIO POR FASES

Filtro gaussiano seguido de detecção de bordas, sobre uma imagem de 16 Mpixels:

Fase / função	Tempo (s)	% total	Classificação
1. le_imagem	0.0251 s	2.9%	[sequencial: E/S de disco]
2. converte_escala_cinza	0.0283 s	3.3%	[paralelizável: pixel a pixel]
3. aplica_filtro_gaussiano	0.7556 s	86.7%	[paralelizável: HOTSPOT]
4. aplica_filtro_sobel	0.0239 s	2.7%	[paralelizável: estêncil 3x3]
5. calcula_estatisticas	0.0155 s	1.8%	[paralelizável: redução]
6. grava_imagem	0.0164 s	1.9%	[sequencial: E/S de disco]

Fração sequencial (s)	: 0.0415 s (4.8%) ==> s = 0.0476		
Fração paralelizável (p)	: 0.8232 s (94.4%) ==> p = 0.9445		
com 8 cores : 6.04x	com 16 cores : 9.38x	com infinitos : 21.00x	

CONSEQUÊNCIA PARA O PROJETO

Uma única fase concentra **86,7%** do tempo. As outras cinco só se tornam relevantes depois que essa for tratada.

LIMITE SUPERIOR DE GANHO

A classificação de cada fase em sequencial ou paralelizável fornece s e p , e com eles o teto de Amdahl: **6,04×** com 8 cores.

PERFIL POR INSTRUMENTAÇÃO: `gprof`

O mesmo programa sob o `gprof`, que desconhece a divisão em fases:

Each sample counts as 0.01 seconds.

% time	cumulative seconds	self seconds	calls	self ms/call	total ms/call	name
88.24	0.60	0.60	1	600.00	600.00	aplica_filtro_gaussiano
4.41	0.63	0.03	1	30.00	30.00	converte_escala_cinza
2.94	0.65	0.02	1	20.00	20.00	aplica_filtro_sobel
2.94	0.67	0.02	1	20.00	20.00	calcula_estatisticas
1.47	0.68	0.01	1	10.00	10.00	save_image_pgm

INTERPRETAÇÃO DAS COLUNAS

`self` é o tempo *dentro* da função, sem o das que ela chama: é essa coluna que identifica o alvo.

`cumulative` mostra que as duas primeiras já somam 92,6%. `calls` é contagem exata, não amostra.

DUAS FONTES DE DISTORÇÃO

Sem `-fno-inline`, o `-O3` funde as funções pequenas e o tempo delas é atribuído à função que as absorveu. E o `-pg` cobra por chamada: funções curtas e muito chamadas aparecem mais custosas do que de fato são.

PERFIL POR AMOSTRAGEM: `perf`

O mesmo programa sob o `perf`, sem recompilação:

```
# Samples: 104 of event 'cpu_core/cycles/P'
# Overhead Command      Shared Object      Symbol
 61.08% filtro_imagem    filtro_imagem      [.] aplica_filtro_gaussiano
 15.35% filtro_imagem    libm.so.6          [.] __sin_fma
  7.13% filtro_imagem    libc.so.6          [.] __memmove_avx_unaligned_erms
  4.35% filtro_imagem    filtro_imagem      [.] generate_test_image
  2.04% filtro_imagem    filtro_imagem      [.] converte_escala_cinza
  1.03% filtro_imagem    [kernel.kallsyms]  [k] 0xffffffff85ea6563
```

CUSTO INVISÍVEL À INSTRUMENTAÇÃO

Os 7,13% em `__memmove` correspondem ao `memcpy` entre os passos do filtro gaussiano. Reside na libc, que não foi compilada com `-pg`: no relatório anterior esse tempo não aparecia.

EFEITO DE UMA ENTRADA NÃO REPRESENTATIVA

Nesta execução a imagem de entrada ainda não existia: `generate_test_image` e `__sin_fma` somam **19,7%** do tempo, gasto na *criação* da entrada e não em seu processamento.

FERRAMENTAS DE GRAFO DE CHAMADAS

O perfil diz quanto cada função custa; o grafo de chamadas diz como elas se encadeiam:

index	% time	self	children	called	name
					<spontaneous>
[1]	100.0	0.00	0.66		main [1]
		0.59	0.00	1/1	aplica_filtro_gaussiano [2]
		0.03	0.00	1/1	converte_escala_cinza [3]
		0.02	0.00	1/1	aplica_filtro_sobel [4]
		0.00	0.00	14/14	get_time_seconds [6]

		0.59	0.00	1/1	main [1]
[2]	89.4	0.59	0.00	1	aplica_filtro_gaussiano [2]

ESTÁTICOS, DO CÓDIGO-FONTE

`cflow`, `doxygen` com `CALL_GRAPH=YES`, e o LLVM com `opt -passes=dot-callgraph`.

Mostram todos os caminhos possíveis, inclusive os que a execução nunca percorre.

DINÂMICOS, DE UMA EXECUÇÃO

`gprof -q`, que atribui tempo a cada aresta; `callgrind` com `gprof2dot`, que dá contagem exata sem recompilar.

Mostram só o que aquela entrada exercitou.

O grafo de chamadas responde **quem chama quem**, não **o que pode executar junto**. Essa segunda pergunta é do compilador: `clang -Rpass-analysis=loop-vectorize`.

PADRÕES FAVORÁVEIS E DESFAVORÁVEIS À PARALELIZAÇÃO

Padrões que se repetem nos trechos de maior custo, e o que cada um exige:

CANDIDATOS DIRETOS

- Varredura de vetor ou matriz em que cada posição sai dos dados de entrada
- A mesma operação sobre casos independentes: parâmetros, arquivos, regiões do domínio
- Acumulação **associativa**: soma, máximo, contagem, em que a ordem não importa
- Busca em um espaço divisível em regiões disjuntas

EXIGEM REFORMULAÇÃO DO ALGORITMO

- **Recorrência**: cada passo consome o resultado do anterior, como uma soma acumulada
- Estado global lido e atualizado a cada iteração
- A ordem de execução faz parte do resultado
- Leitura e escrita de arquivo, que o disco serializa

A coluna da direita não indica impossibilidade: o custo está na **reformulação do algoritmo**, não na distribuição do trabalho existente.

CRITÉRIOS DE SELEÇÃO DO CANDIDATO

Um bom candidato à paralelização precisa das **duas** propriedades:

CUSTO ELEVADO

Consumir fração significativa do tempo total. É o perfil que responde.

TRABALHO INDEPENDENTE

As iterações não podem depender umas das outras. É a leitura do código que responde.

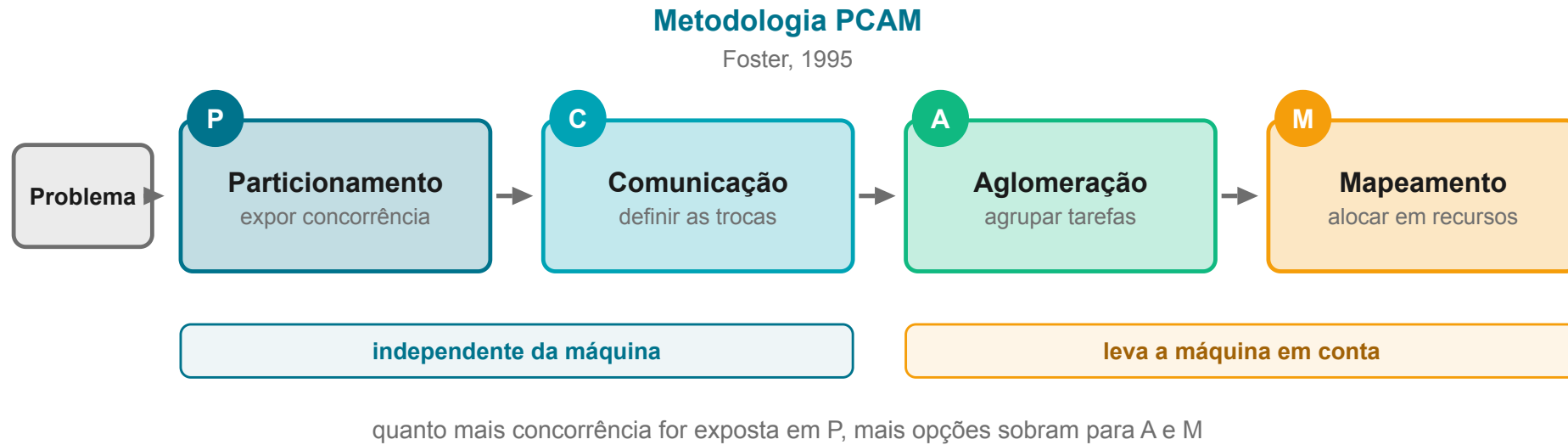
Custo elevado, mas com dependência carregada entre iterações? Então o trabalho é reformular o algoritmo, não inserir uma diretiva.

Com o candidato identificado, começa a modelagem propriamente dita.

PCAM

Uma metodologia para projetar programas paralelos

AS QUATRO ETAPAS

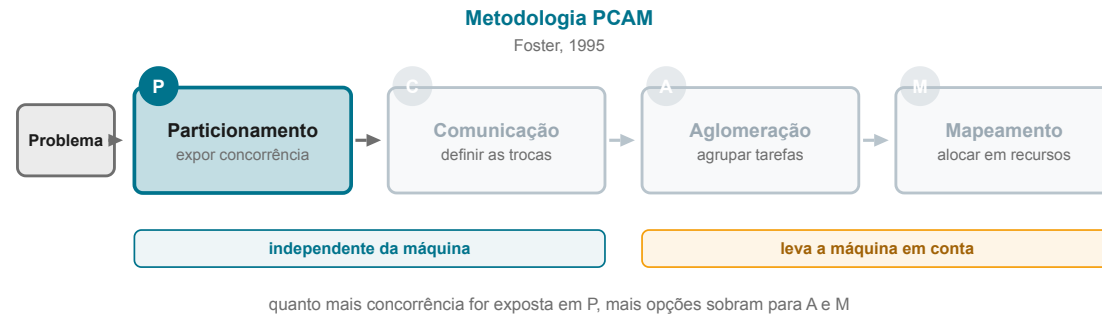


As duas primeiras etapas são **independentes da máquina**: exponha o máximo de concorrência sem pensar em quantos cores você tem. A máquina só entra em A e M.

P: PARTICIONAMENTO

Expor o máximo de concorrência

PARTICIONAMENTO: O OBJETIVO



O QUE FAZER

Quebrar o problema no **maior número possível** de tarefas pequenas, ignorando quantos processadores existem.

O QUE NÃO FAZER

Já pensar em "tenho 8 cores, logo 8 tarefas". Isso **fecha portas** que a aglomeração abriria depois.

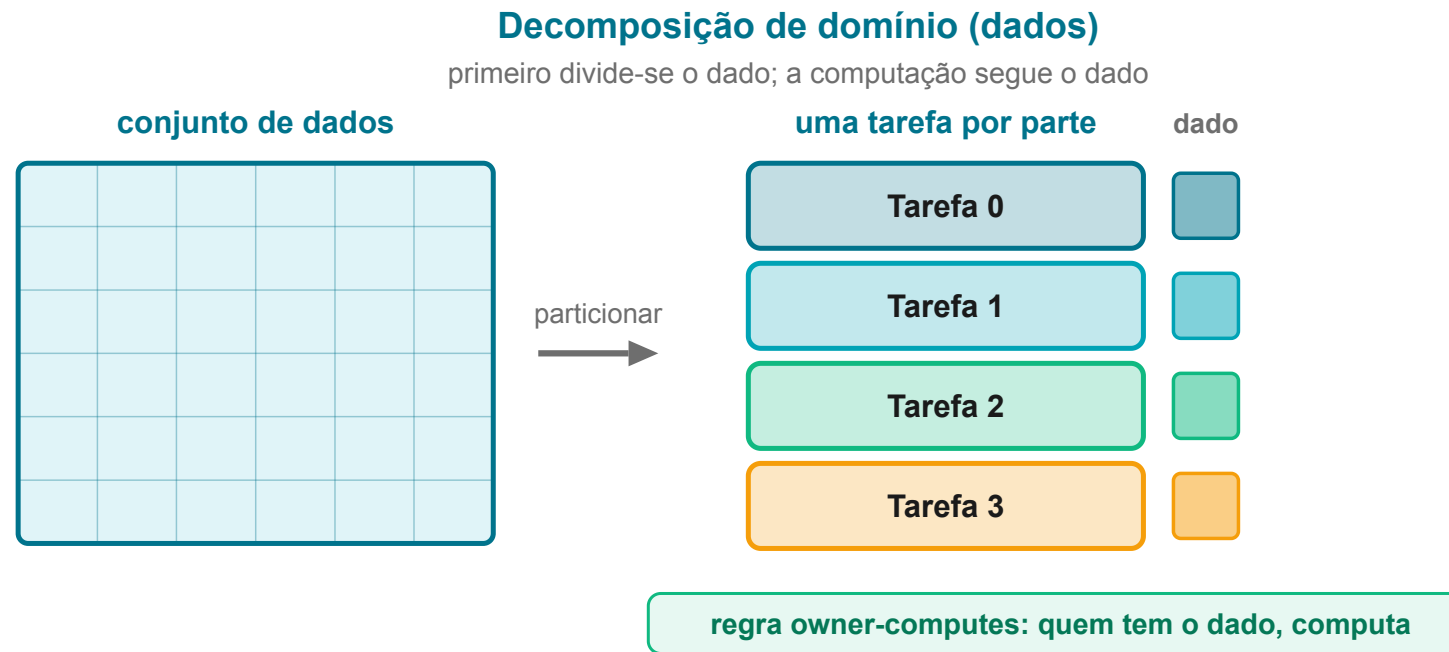
Regra prática: gere uma ordem de grandeza a mais de tarefas do que processadores.

TAREFA ≠ THREAD ≠ PROCESSO ≠ CORE

Conceito	Onde vive	O que é
Tarefa	modelagem	Unidade lógica de trabalho. Existe no seu projeto, não no sistema operacional
Thread	so	Fluxo de execução que <i>compartilha</i> memória com os irmãos
Processo	so	Fluxo de execução com espaço de endereçamento <i>próprio</i>
Core	hardware	Unidade física que executa. Você não controla, você pede

Você decompõe em **tarefas**. O mapeamento é que decide quantas threads ou processos vão executá-las, e o sistema operacional decide em quais cores.

DECOMPOSIÇÃO DE DOMÍNIO (DADOS)



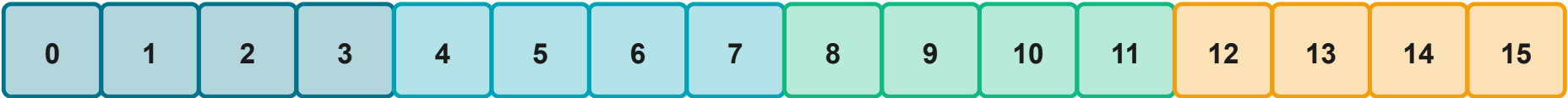
Primeiro divide-se o **dado**; a computação vai atrás. É a técnica mais usada em computação científica.

COMO DIVIDIR: DISTRIBUIÇÕES 1D

Distribuições 1D

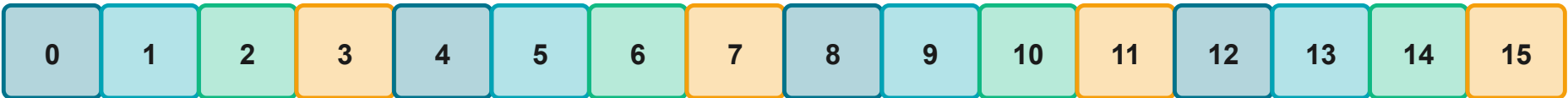
vetor de 16 elementos entre 4 processos

Blocos



cada processo recebe uma faixa contígua — boa localidade, ruim se o custo variar ao longo do vetor

Cíclica



os elementos são intercalados — equilibra custo variável, mas espalha os acessos à memória

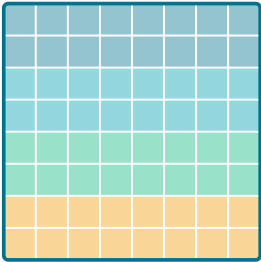


DISTRIBUIÇÕES 2D

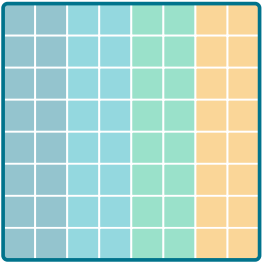
Distribuições 2D

matriz 8×8 entre 4 processos — a notação diz o que se faz em cada dimensão

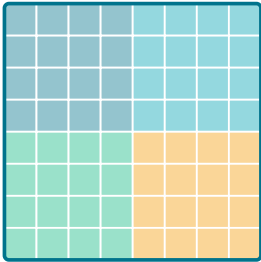
(Bloco, *)



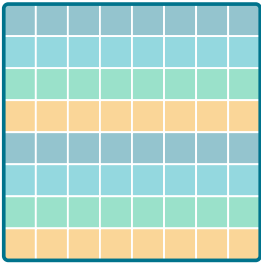
(*, Bloco)



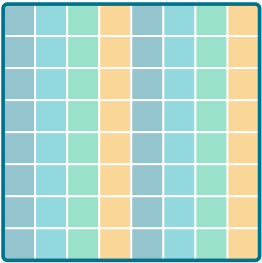
(Bloco, Bloco)



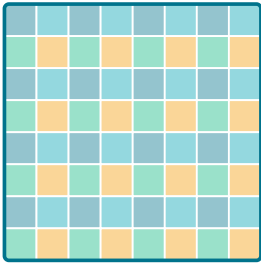
(Cíclica, *)



(*, Cíclica)



(Cíclica, Cíclica)



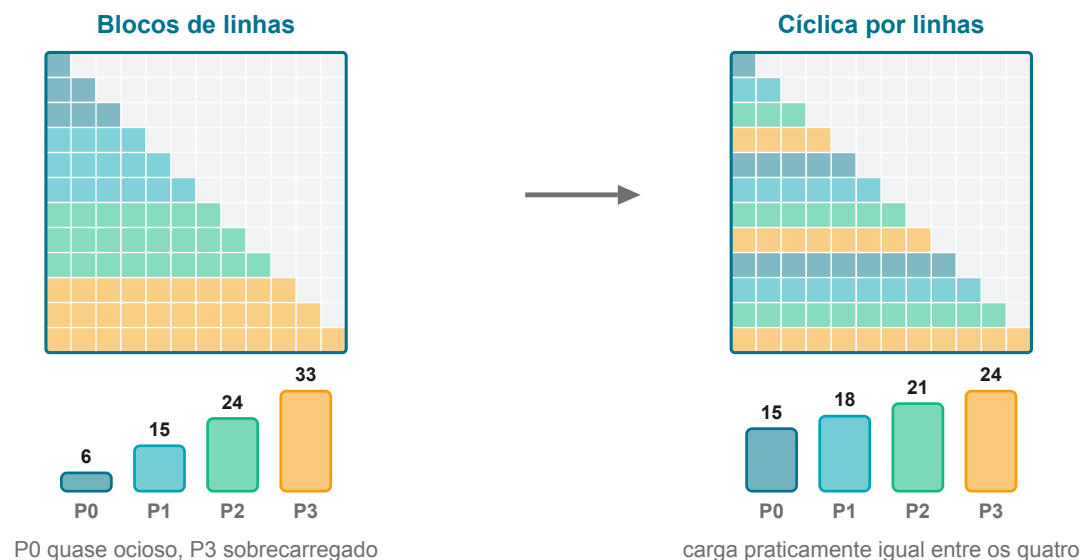
o par (linhas, colunas) define a estratégia em cada dimensão — cores indicam o processo dono

QUANDO A DISTRIBUIÇÃO DECIDE O BALANCEAMENTO

Na matriz triangular a linha i tem i elementos: o custo por linha **cresce**. Mesmo total de trabalho, mesmos quatro processos: só muda quem recebe quais linhas.

Quando a distribuição decide o balanceamento

matriz triangular: só a parte preenchida tem trabalho

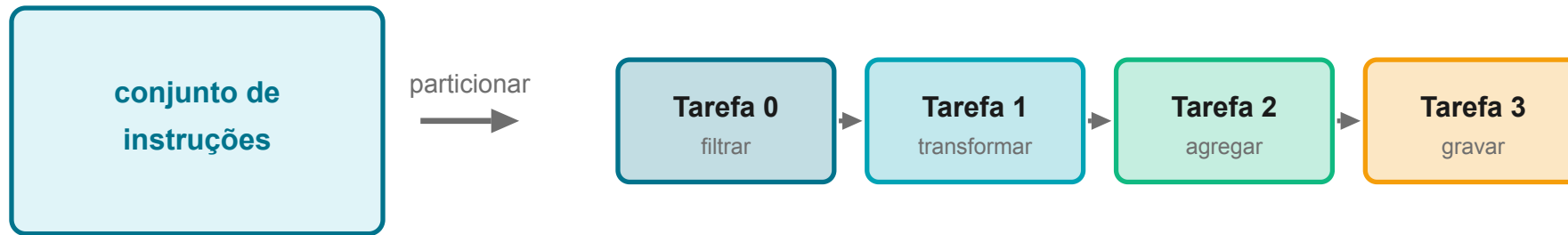


O programa termina com o processo mais lento: 33 unidades com blocos, 24 com cíclica. 27% mais rápido sem mudar uma linha do algoritmo.

DECOMPOSIÇÃO FUNCIONAL (TAREFAS)

Decomposição funcional (tarefas)

primeiro divide-se a computação; os dados fluem entre as partes



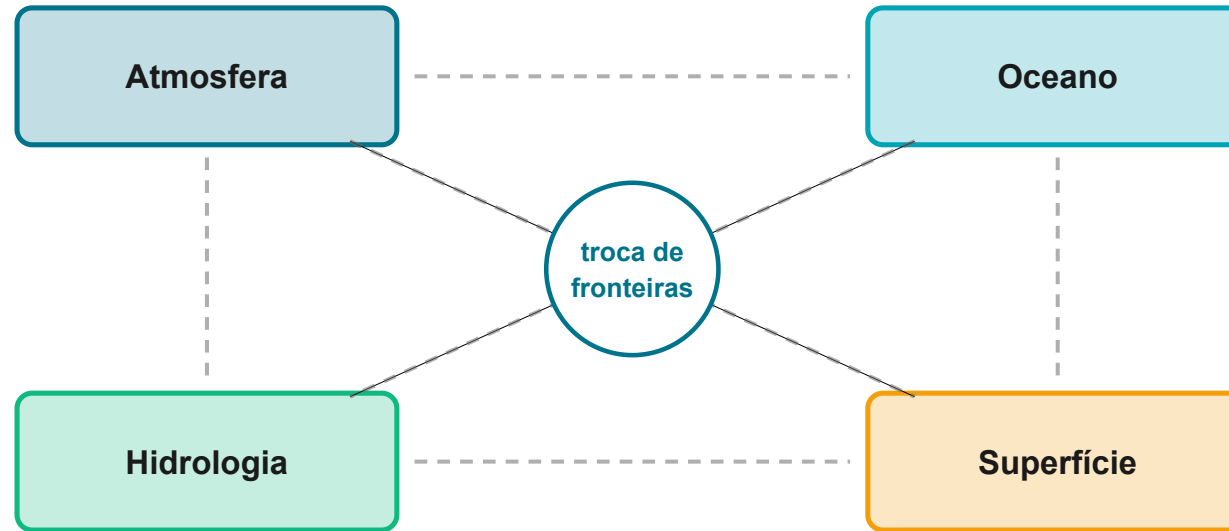
cada tarefa faz algo diferente — o dado é que se move de uma para a outra
se as tarefas precisam dos mesmos dados, provavelmente a decomposição de domínio é melhor

Aqui a concorrência vem da **estrutura do problema**, não do tamanho do dado.

EXEMPLO: MODELO CLIMÁTICO ACOPLADO

Exemplo: modelo climático acoplado

quatro modelos distintos, executando ao mesmo tempo



a concorrência vem da estrutura do problema, não da divisão de um vetor

DOMÍNIO OU FUNCIONAL?

Pergunta	Decomposição de domínio	Decomposição funcional
O que se divide	Os dados	As operações
Quantas tarefas	Cresce com o tamanho do problema	Fixo, limitado pelo número de etapas
Escala?	sim : mais dados, mais tarefas	não : o teto é o número de funções
Quando usar	Padrão em problemas científicos	Fluxos, pipelines, sistemas acoplados

Na dúvida, **comece pelo domínio**: é a única que escala com o tamanho do problema. Depois verifique se há decomposição funcional aproveitável por cima.

DECOMPOSIÇÃO RECURSIVA

A IDEIA

Dividir o problema em subproblemas do **mesmo tipo**, resolver cada um independentemente e combinar os resultados.

Cada nível da recursão dobra o número de tarefas disponíveis.

ONDE APARECE

- *Quicksort* e *mergesort*
- Busca de mínimo e máximo
- Multiplicação de matrizes por blocos
- Simulação de N corpos (Barnes-Hut)

A profundidade da recursão é um **parâmetro de granularidade**: parar cedo demais gera poucas tarefas; parar tarde demais gera overhead que domina o trabalho.

OUTRAS FORMAS DE DECOMPOR

EXPLORATÓRIA

Dividir um **espaço de busca** entre as tarefas: quebra-cabeças, jogos, *backtracking*, otimização combinatória.

Curiosidade: o speed-up pode ser *super* ou *sublinear* por sorte, dependendo de qual tarefa encontra a solução primeiro.

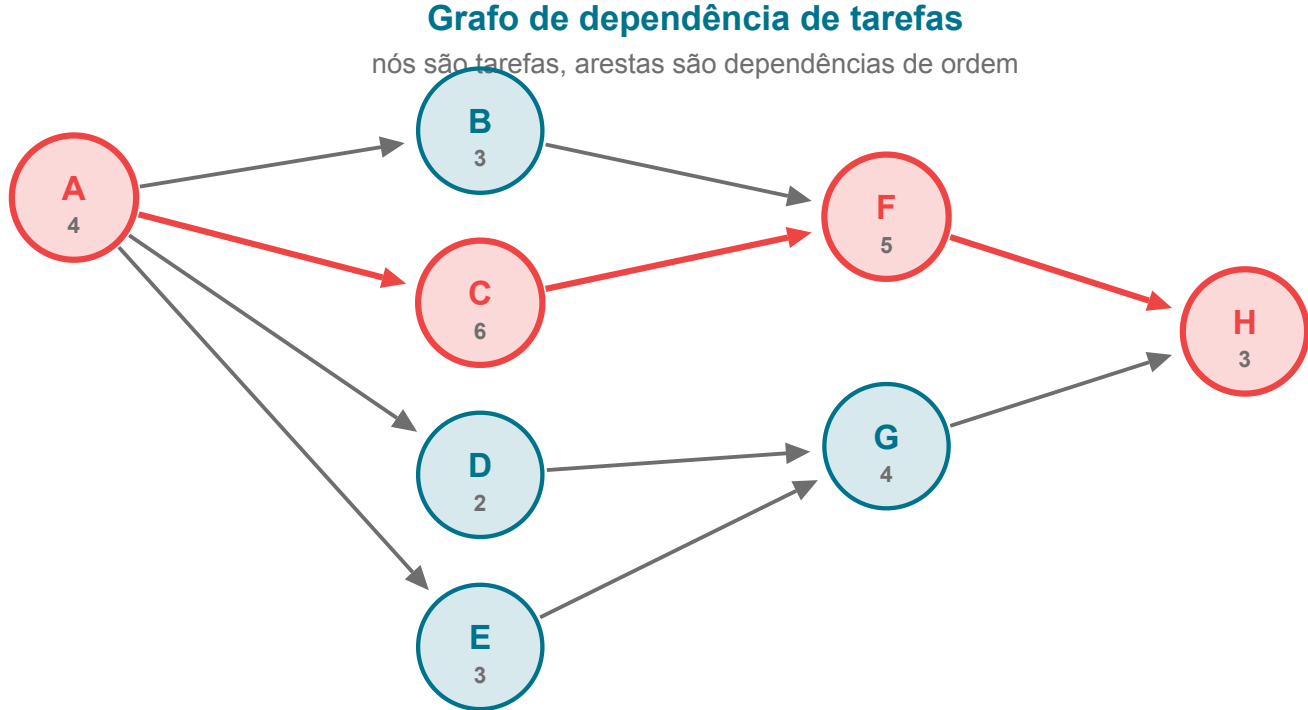
ESPECULATIVA

Executar **vários ramos** antes de saber qual será necessário, e descartar os errados.

Troca trabalho desperdiçado por tempo. Só compensa quando há recursos ociosos.

Nesta disciplina vocês vão usar **domínio e funcional**, com recursiva aparecendo em alguns algoritmos. Estas duas ficam registradas para reconhecimento.

GRAFO DE DEPENDÊNCIA DE TAREFAS



número no nó =
custo da tarefa

caminho crítico = 4 + 6 + 5 + 3 = 18

trabalho total = 30

$SU \leq 30/18 = 1,7$

O QUE O GRAFO DIZ SOBRE O SPEED-UP

TRABALHO TOTAL

A soma do custo de todas as tarefas. É o tempo **sequencial**.

CAMINHO CRÍTICO

A cadeia de dependências mais longa. É o menor tempo possível com **infinitos** processadores.

GRAU DE CONCORRÊNCIA

Quantas tarefas ficam prontas ao mesmo tempo. Limita quantos processadores fazem sentido.

$$SU_{\text{máx}} = \frac{\text{trabalho total}}{\text{caminho crítico}}$$

É o mesmo teto de Amdahl, visto pelo lado do grafo: o caminho crítico é a parte que não dá para paralelizar.

CHECKLIST DO PARTICIONAMENTO

Antes de passar para a próxima etapa, verifique:

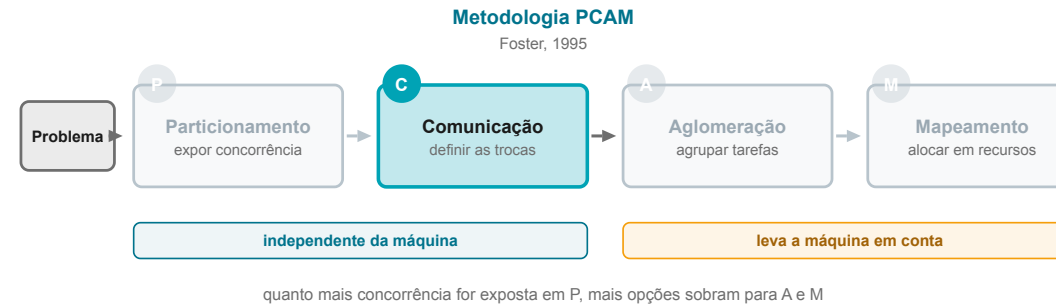
- ✓ Foram definidas **mais tarefas do que processadores** da máquina alvo?
- ✓ O particionamento evita **redundância** de cálculo e de acesso à memória?
- ✓ As tarefas têm **tamanho parecido** entre si?
- ✓ O **número de tarefas cresce** com o tamanho do problema?
- ✓ Foram consideradas **partições alternativas**?

A quarta pergunta é a que separa um programa que funciona hoje de um que ainda funciona no cluster do ano que vem.

C: COMUNICAÇÃO

Que dados precisam atravessar as fronteiras das tarefas

COMUNICAÇÃO: O SEGUNDO GRAFO



GRAFO DE DEPENDÊNCIA

Diz **ordem**: quem precisa terminar antes de quem começar.

Determina o teto do speed-up.

GRAFO DE INTERAÇÃO

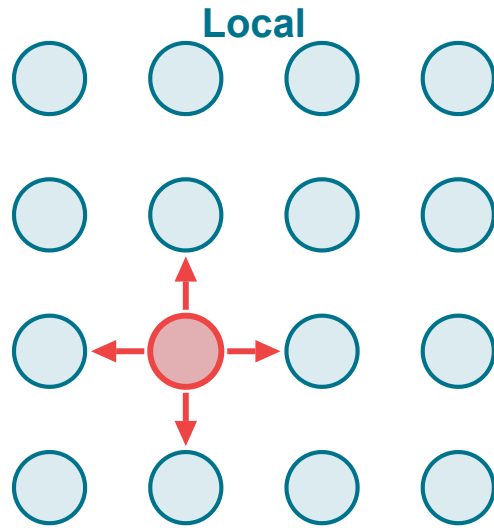
Diz **troca de dados**: quem precisa de qual pedaço de quem.

Determina o custo de comunicação.

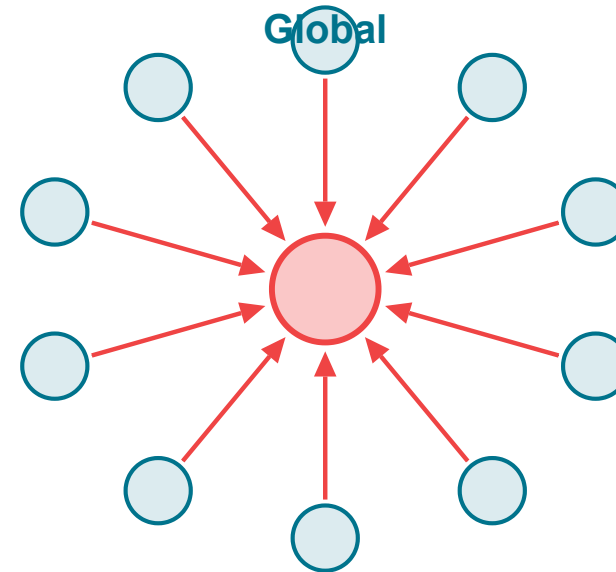
São dois grafos diferentes sobre as mesmas tarefas, e eles levam a decisões diferentes nas etapas seguintes.

COMUNICAÇÃO LOCAL VS. GLOBAL

Comunicação local vs. global



cada tarefa fala só com vizinhos
custo não cresce com p

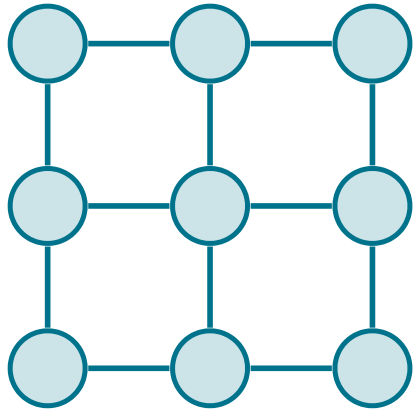


uma tarefa depende de todas
custo cresce com p

ESTRUTURADA VS. NÃO ESTRUTURADA

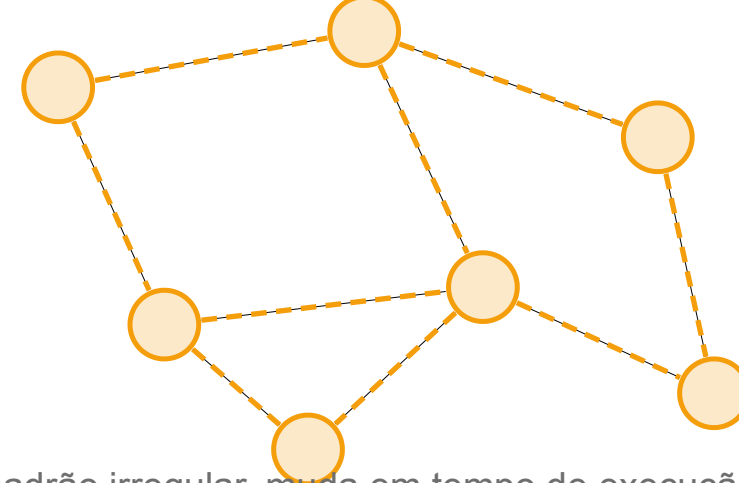
Estruturada e estática vs. não estruturada e dinâmica

Estruturada e estática



padrão regular, conhecido antes de executar
o compilador e o programador podem otimizar

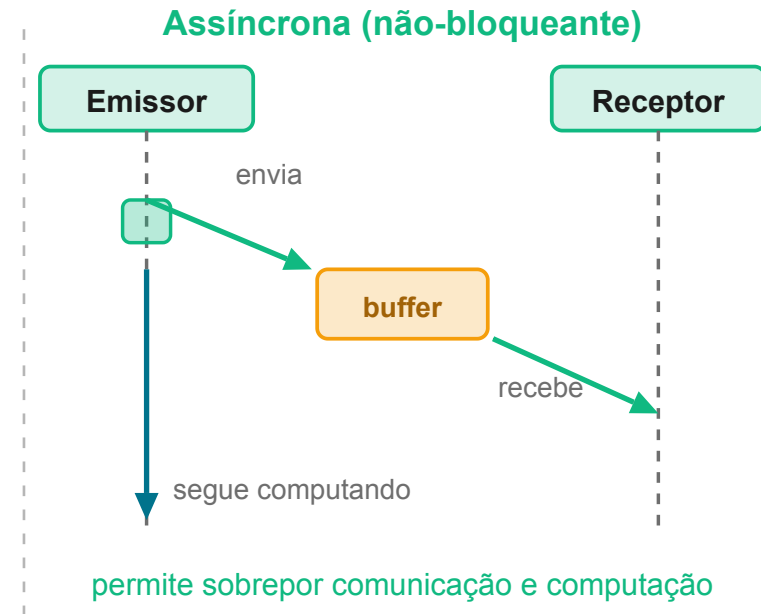
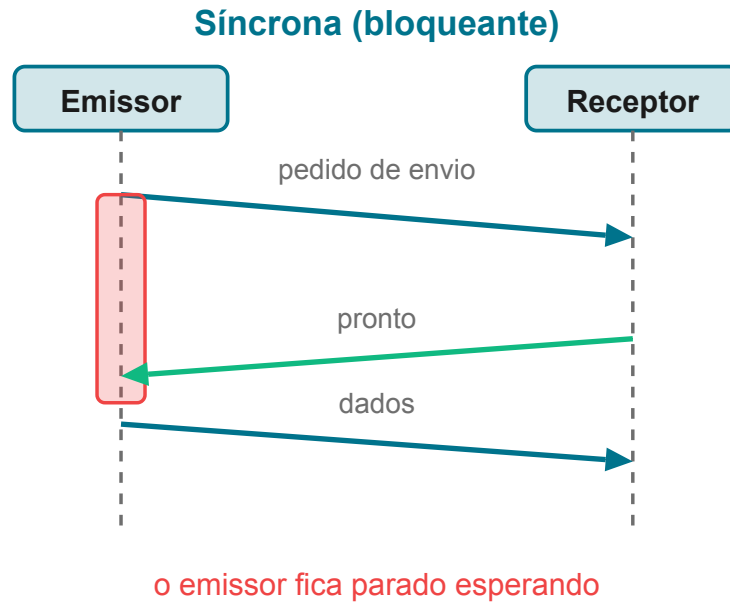
Não estruturada e dinâmica



padrão irregular, muda em tempo de execução
exige balanceamento dinâmico

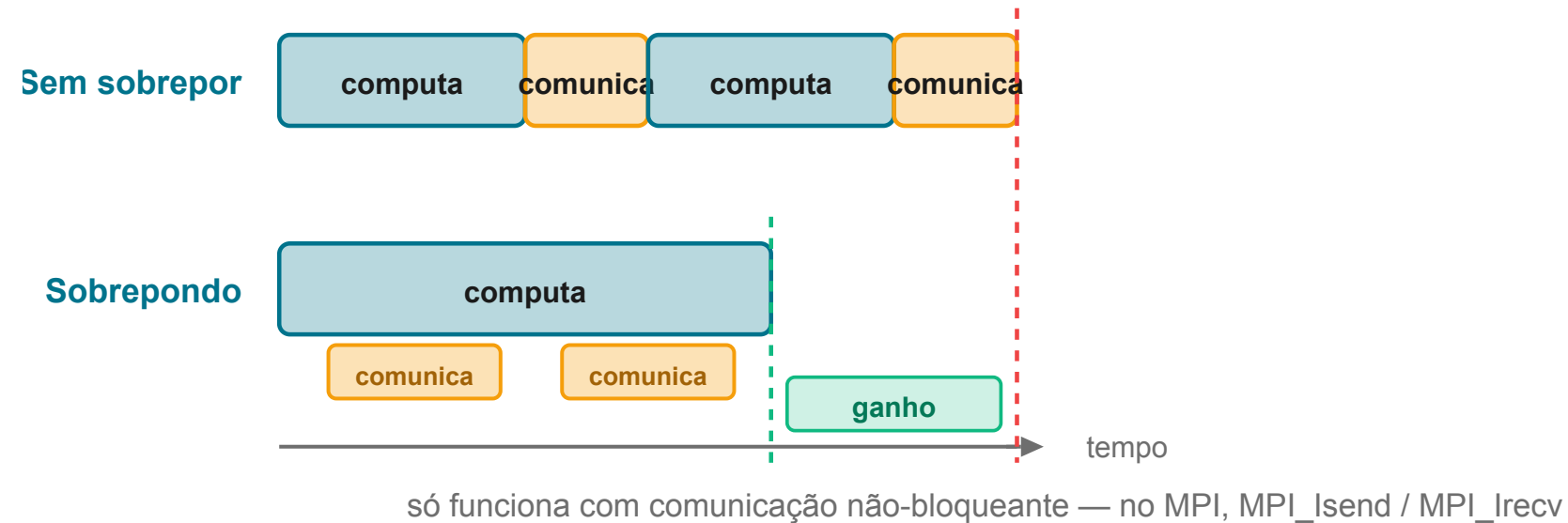
SÍNCRONA VS. ASSÍNCRONA

Comunicação síncrona vs. assíncrona



SOBREPOR COMUNICAÇÃO E COMPUTAÇÃO

Sobrepор comunicação e computação



A comunicação que você consegue esconder atrás da computação é **de graça**.

EM MEMÓRIA COMPARTILHADA, TAMBÉM HÁ COMUNICAÇÃO

Não existe `send`, mas existe **tráfego**. A comunicação é implícita; o custo não é.

COERÊNCIA DE CACHE

Escrever em uma variável que outro core leu **invalida** a linha na cache dele. O protocolo custa tempo.

FALSE SHARING

Duas threads escrevem em variáveis *diferentes* na **mesma linha de cache**. Não há corrida, mas há tráfego constante.

NUMA

Acessar memória de outro soquete custa mais. Onde a página foi alocada vira decisão de desempenho.

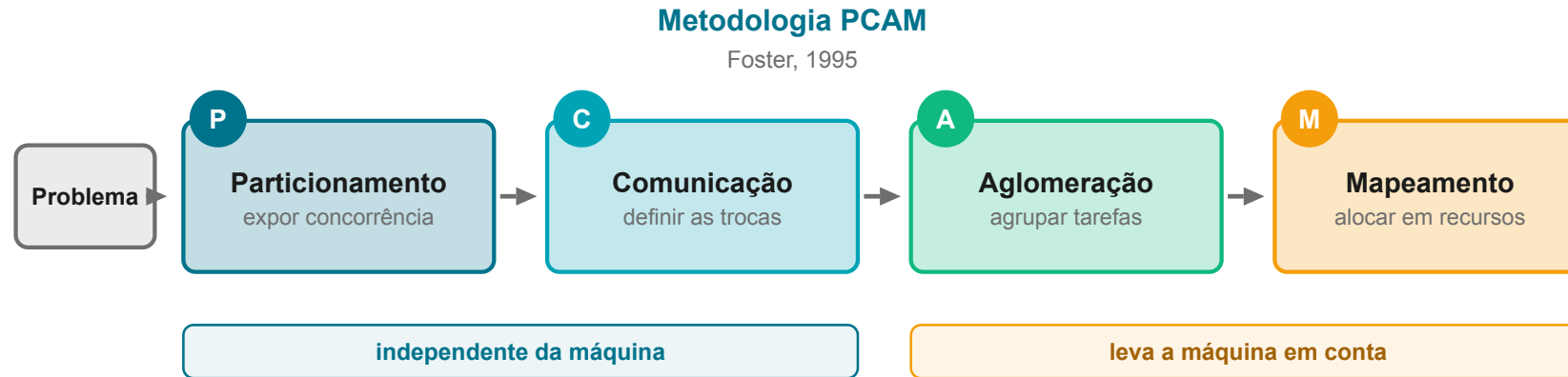
Tudo isso é **hardware**, visto na Unidade 01. Aqui vira **problema de programação**.

CHECKLIST DA COMUNICAÇÃO

- ✓ Todas as tarefas realizam **aproximadamente o mesmo número** de operações de comunicação?
- ✓ Cada tarefa se comunica apenas com um **pequeno número de vizinhos**?
- ✓ As operações de comunicação podem ocorrer **concorrentemente** entre si?
- ✓ A comunicação pode ser **sobreposta** à computação?

Se uma tarefa se comunica com *todas* as outras, volte ao particionamento. Nenhuma etapa posterior conserta isso.

METADE DO CAMINHO



P E C: COBERTAS

Concorrência exposta e trocas identificadas, **sem olhar para a máquina.**

A E M: A SEGUIR

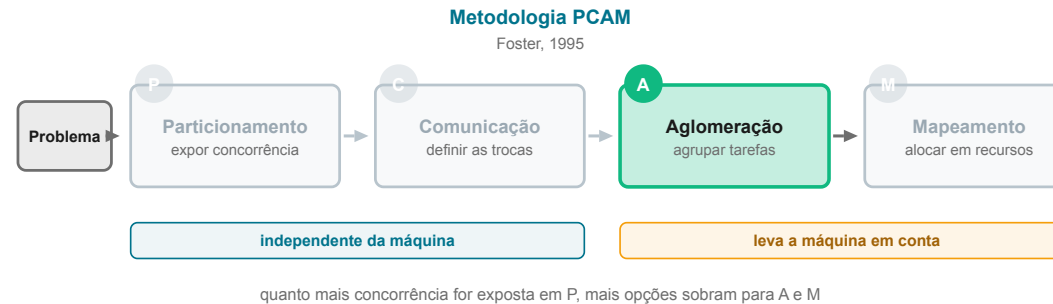
Agora a máquina entra: quantos cores, que rede, que latência.

Exercício: pegue o programa sequencial do seu trabalho, profile com `gprof` e desenhe o grafo de dependência do trecho mais quente.

A: AGLOMERAÇÃO

Quando parar de dividir

AGLOMERAÇÃO: O OBJETIVO



Agrupar tarefas pequenas em tarefas maiores, **sacrificando parte do paralelismo exposto** para reduzir comunicação e overhead.

O QUE SE GANHA

Menos mensagens, melhor localidade de dados, menos custo de criar e gerenciar tarefas.

O QUE SE PERDE

Flexibilidade de mapeamento e capacidade de balancear a carga.

GRANULARIDADE

Quantidade de trabalho por tarefa. Poucas tarefas grandes, ou muitas pequenas?

Grão	Trabalho por tarefa	Arquitetura adequada	Risco
Fino	Poucas operações	SIMD, GPU, dentro do core	Overhead domina
Médio	Um laço, um bloco	Multicore, memória compartilhada	Equilíbrio delicado
Grosso	Um subdomínio inteiro	Cluster, troca de mensagens	Pouco paralelismo, desbalanceio

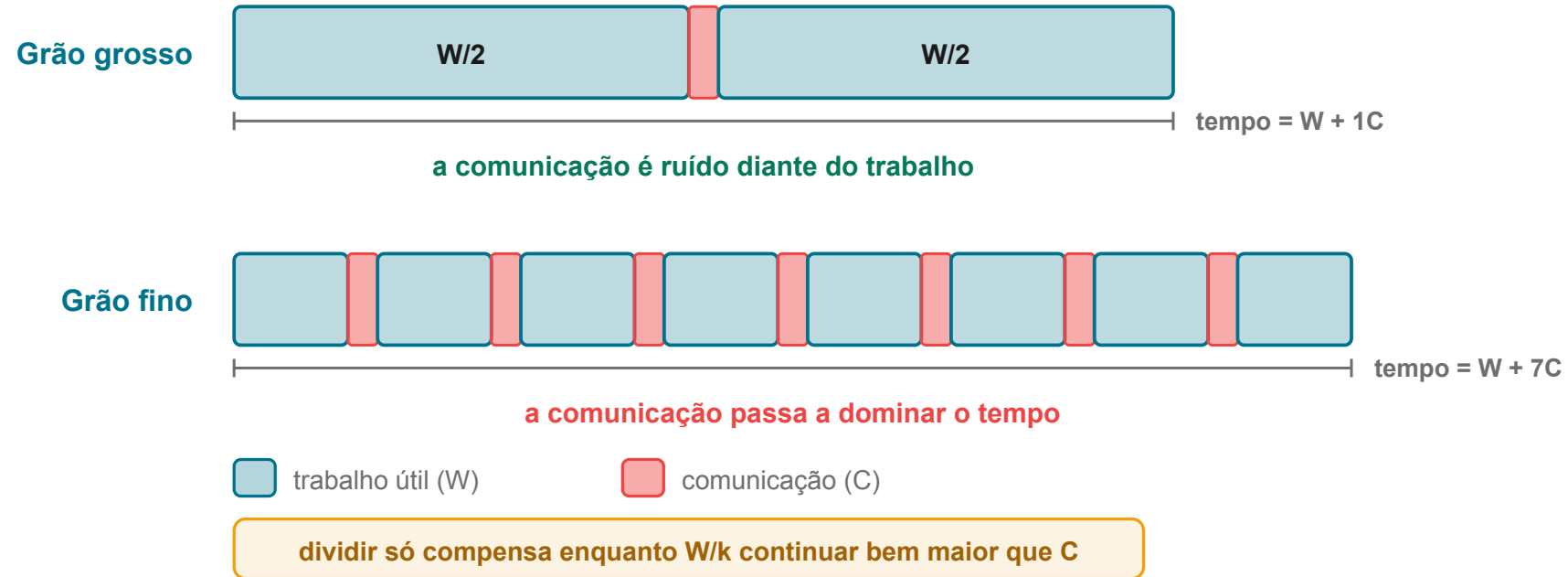
Regra prática: o grão precisa ser maior que o custo de criá-lo e alimentá-lo.

A latência da rede é *ordens de grandeza* maior que a de memória. É por isso que cluster exige grão grosso.

O MESMO TRABALHO, DUAS DIVISÕES

Granularidade: o mesmo trabalho, duas divisões

W = trabalho por tarefa · C = custo de comunicação



W E C DEPENDEM DA MÁQUINA

W: TRABALHO

Depende da **capacidade de processamento do nó**: frequência, número de cores, largura das unidades vetoriais.

Um nó mais rápido executa o mesmo grão em menos tempo: e o grão fica relativamente *menor*.

C: COMUNICAÇÃO

Depende da **latência e da vazão** da rede de interconexão.

InfiniBand e Ethernet de 1 Gb levam a granularidades ótimas completamente diferentes.

A granularidade ótima é propriedade do **par aplicação-máquina**, nunca da aplicação sozinha.

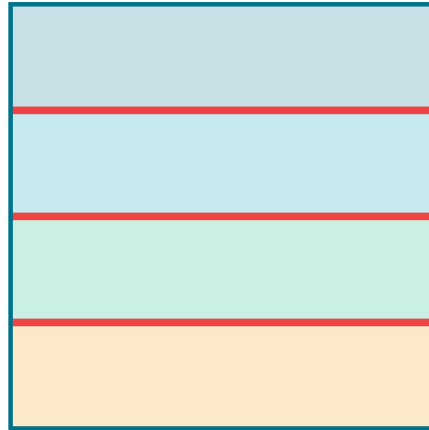
Corolário desagradável: um programa afinado para um cluster pode ficar mal afinado ao mudar de cluster. É por isso que o tamanho do bloco costuma virar parâmetro de execução.

SUPERFÍCIE E VOLUME

Superfície e volume: faixas vs. blocos

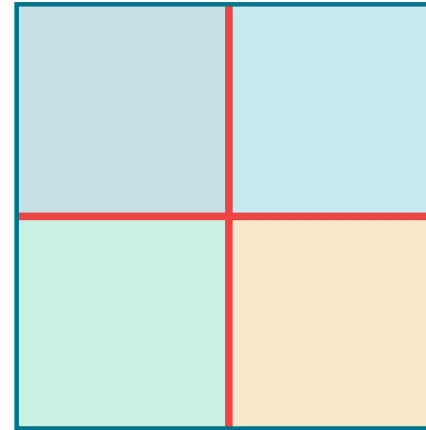
mesmo domínio $N \times N$, mesmos 4 processos — fronteiras diferentes

Faixas (1D)



fronteira interna = $3 \times N$

Blocos (2D)



fronteira interna = $2 \times N$

com p processos: faixas comunicam $\sim N \cdot p$, blocos $\sim N \cdot \sqrt{p}$ — a vantagem cresce com p

DUAS FORÇAS OPOSTAS

AGLOMERAR MAIS

Criar e gerenciar tarefa **custa**. Um milhão de tarefas de dez operações cada gasta mais em gerência do que em trabalho.

Menos tarefas, maiores, com menos comunicação entre elas.

AGLOMERAR MENOS

Superdecomposição: manter mais tarefas do que cores, de propósito.

É o que permite balanceamento dinâmico e o que faz o programa aproveitar uma máquina maior sem recompilar.

O ponto de equilíbrio: tarefas **grandes o bastante** para o overhead ser irrelevante, e **numerosas o bastante** para haver o que redistribuir.

Um bom alvo é da ordem de **10 tarefas por core**: o suficiente para balancear sem pagar caro por gerência.

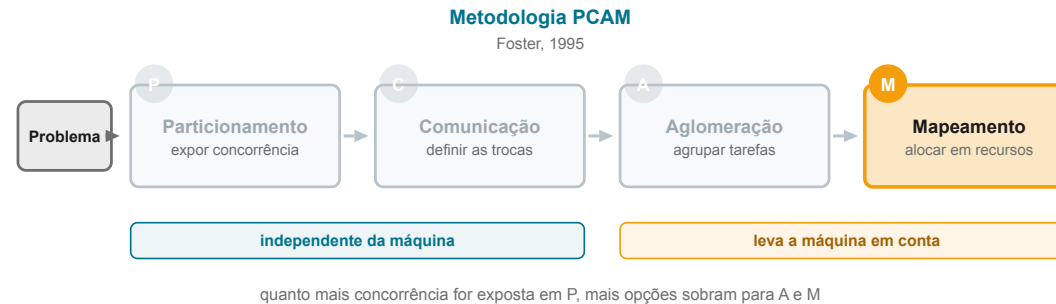
CHECKLIST DA AGLOMERAÇÃO

- ✓ A aglomeração **reduziu o custo de comunicação** e aumentou a localidade dos dados?
- ✓ Se houve **replicação de computação**, o benefício superou o custo?
- ✓ As tarefas resultantes têm **custo de computação e de comunicação similares** entre si?
- ✓ O número de tarefas **ainda escala** com o tamanho do problema?
- ✓ O custo de **modificar o código sequencial** foi razoável?

M: MAPEAMENTO

Quem executa o quê, e onde

MAPEAMENTO: O OBJETIVO



Decidir em qual processador cada tarefa executa: a última etapa, e a única que o sistema operacional pode desfazer por você.

DOIS OBJETIVOS

Reduzir a **ociosidade** (balancear a carga) e reduzir a **comunicação** (preservar a localidade).

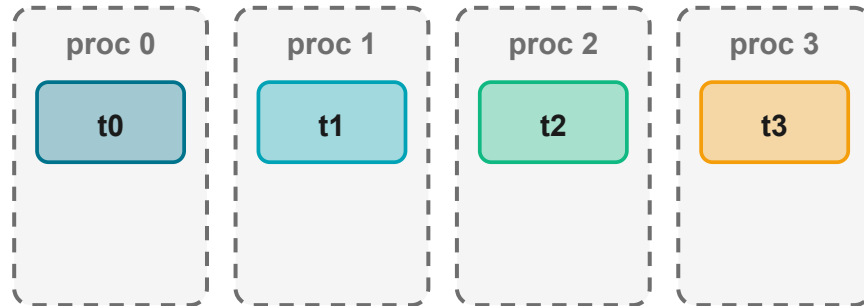
UM PROBLEMA

Eles se contradizem: e o caso geral é **NP-difícil**.
Trabalha-se com heurísticas.

O CONFLITO CENTRAL

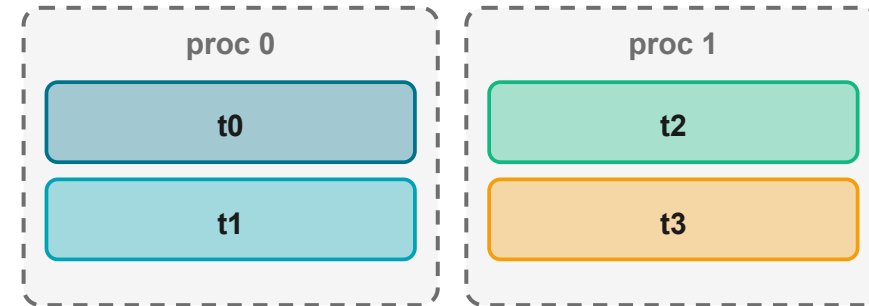
O conflito central do mapeamento

Espalhar: 4 processadores



concorrência máxima, muita comunicação

Agrupar: 2 processadores



menos comunicação, menos paralelismo

melhorar um dos dois quase sempre piora o outro

DE ONDE VEM O DESBALANCEAMENTO

DA APLICAÇÃO

Custo por tarefa que varia com o dado: matriz triangular, malha adaptativa, *Mandelbrot*, busca com poda.

DA MÁQUINA

Nós heterogêneos, cores com frequências diferentes, um nó com memória mais lenta.

DO AMBIENTE

Ruído do sistema operacional, outros trabalhos no mesmo nó, interferência de rede.

O programa termina quando o último processo termina. Um processo 30% mais lento atrasa todos os outros em 30%.

A origem do desbalanceamento decide a estratégia: se é **previsível**, mapeamento estático resolve. Se não é, precisa ser dinâmico.

MAPEAMENTO ESTÁTICO

A atribuição é decidida **antes da execução** e não muda.

A FAVOR

- Custo zero em tempo de execução
- Localidade previsível e otimizável
- Comportamento reproduzível

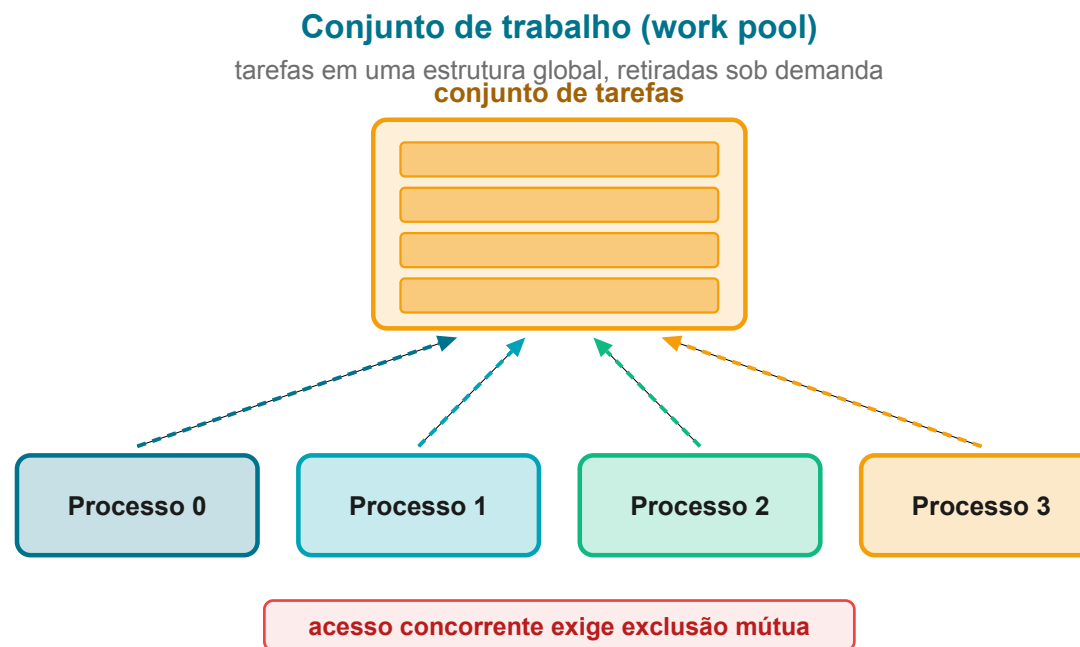
CONTRA

- Exige conhecer o custo das tarefas *a priori*
- Nada a fazer se a estimativa estiver errada
- Não absorve variação do ambiente

Estratégias:

Distribuições de arranjos (blocos, cíclica, bloco-cíclica) · Particionamento de grafos para malhas irregulares · Atribuição aleatória

MAPEAMENTO DINÂMICO: CENTRALIZADO



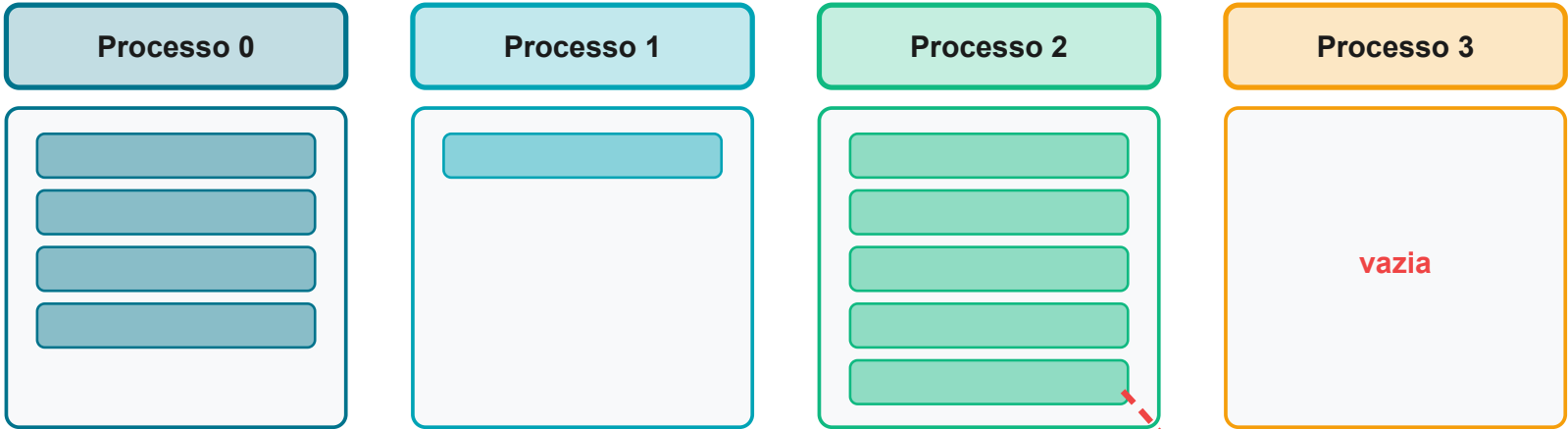
Balanceia sozinho: quem termina antes pega mais trabalho. Não é preciso estimar custo nenhum.

O mestre satura: com muitos processos, todos disputam a mesma estrutura e a coordenação vira o gargalo.

MAPEAMENTO DINÂMICO: DISTRIBUÍDO

Work stealing

cada processo tem sua fila; quem fica ocioso rouba



roubar da extremidade oposta reduz a chance de conflito com o dono da fila

rouba da outra ponta

COMO ESCOLHER

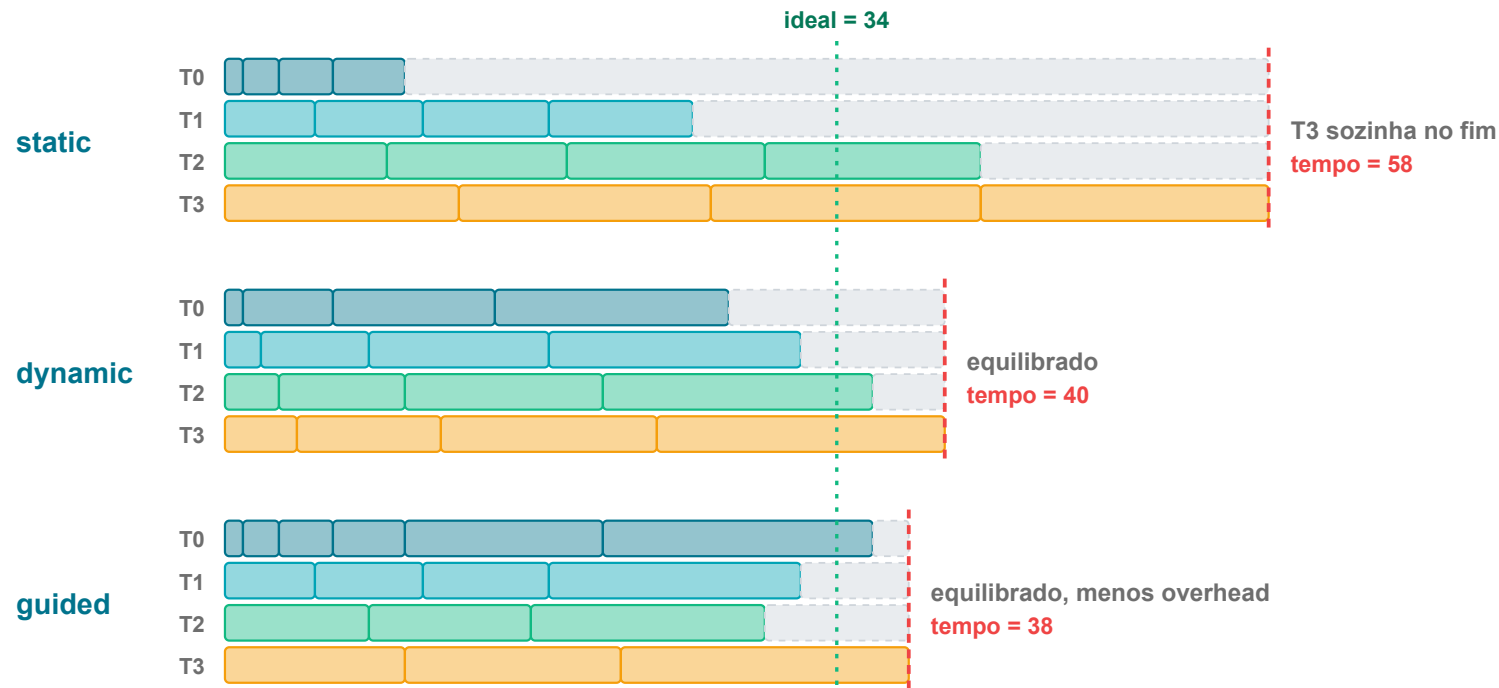
Situação	Escolha	Por quê
Custo por tarefa uniforme e conhecido	estático	Overhead zero, e não há o que balancear
Custo variável mas previsível pelo dado	estático cíclico	Intercalar já equilibra, sem custo em execução
Custo imprevisível	dinâmico	Só medindo em execução dá para equilibrar
Muitos processos, tarefas curtas	work stealing	Evita a contenção de uma fila central
Comunicação intensa entre vizinhos	estático	Dinâmico destruiria a localidade

O mapeamento dinâmico não é gratuito: ele troca **ociosidade** por **comunicação e perda de localidade**.

ESCALONAMENTO DE LAÇOS

Escalonamento de laços: carga triangular

laço de 16 iterações em que a iteração i custa proporcionalmente a i



blocos tracejados = tempo ocioso · a linha vermelha marca o fim do laço

CHECKLIST DO MAPEAMENTO

- ✓ Se o mapeamento é estático, a **carga ficou de fato equilibrada**? Foi medido?
- ✓ Se é dinâmico, o **escalonador não virou gargalo**?
- ✓ Tarefas que **se comunicam muito** ficaram próximas?
- ✓ A **ociosidade** foi medida, e não apenas suposta?

Meça o tempo **por processo**, não só o total. A média esconde exatamente o que você precisa enxergar.

ATIVIDADE: COLORIR 10 IMAGENS

A turma precisa colorir **10 imagens** com desenhos diferentes, no menor tempo possível.

Restrições:

- 1** | As imagens estão todas em uma **mesa separada**, na frente da sala
- 2** | Cada aluno só pode colorir **na sua própria mesa**
- 3** | Existe **apenas uma caneta de cada cor**, e todas ficam na mesa das imagens

Como vocês organizariam o trabalho?

O QUE CADA RESTRIÇÃO REPRESENTA

Na atividade	Na máquina	Consequência de projeto	Etapa
Dividir o trabalho por imagem	decomposição de domínio	Divisão natural: nenhuma imagem depende de outra	P
Ir até a mesa buscar a imagem	custo de comunicação	Vale buscar várias de uma vez: <i>aglomeração</i>	C → A
Só colorir na própria mesa	memória local	O dado precisa ser movido antes de ser usado: é o que cria a comunicação	C
Uma única caneta por cor	recurso compartilhado	Exclusão mútua; quem espera fica ocioso	C
Imagens de complexidade diferente	carga desigual	Distribuição estática desbalanceia: melhor sob demanda	M
Chamar mais alunos que imagens	excesso de recursos	Não acelera: o particionamento fixou o grau de concorrência em 10	M

Nós acabamos de discutir, sem código, as mesmas decisões que o PCAM organiza.

MODELOS DE APLICAÇÃO

Cinco estruturas que se repetem em quase todo programa paralelo

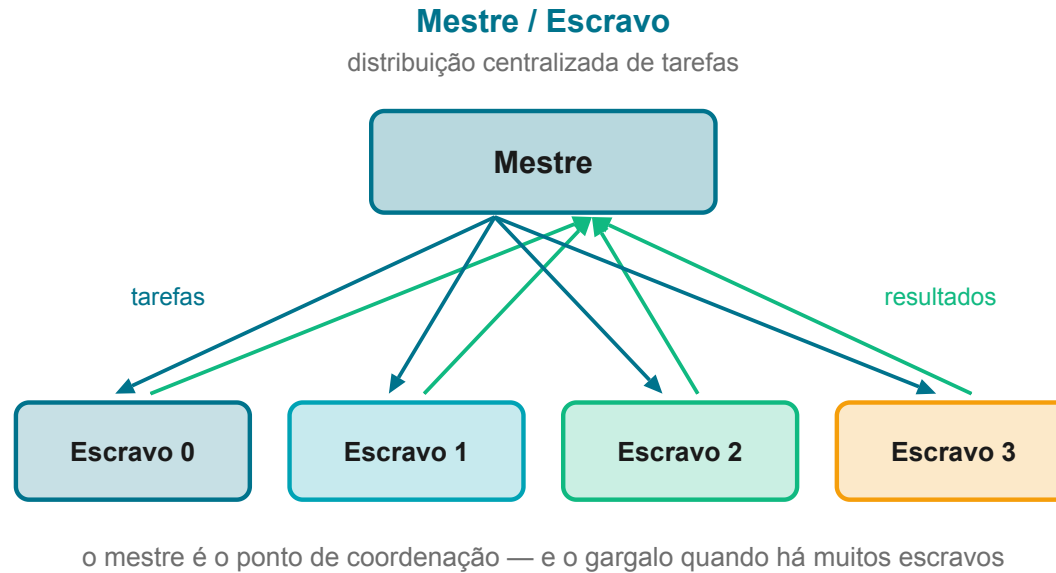
PADRÕES QUE SE REPETEM

Aplicações paralelas são modeladas por um **grafo**: vértices são tarefas, arestas são trocas de dados.

Alguns formatos de grafo aparecem repetidamente. Cinco cobrem a maioria dos casos:

- 1 | Mestre / Escravo
- 2 | Pipeline
- 3 | Conjunto de trabalho (*work pool*)
- 4 | Divisão e conquista
- 5 | Fases paralelas

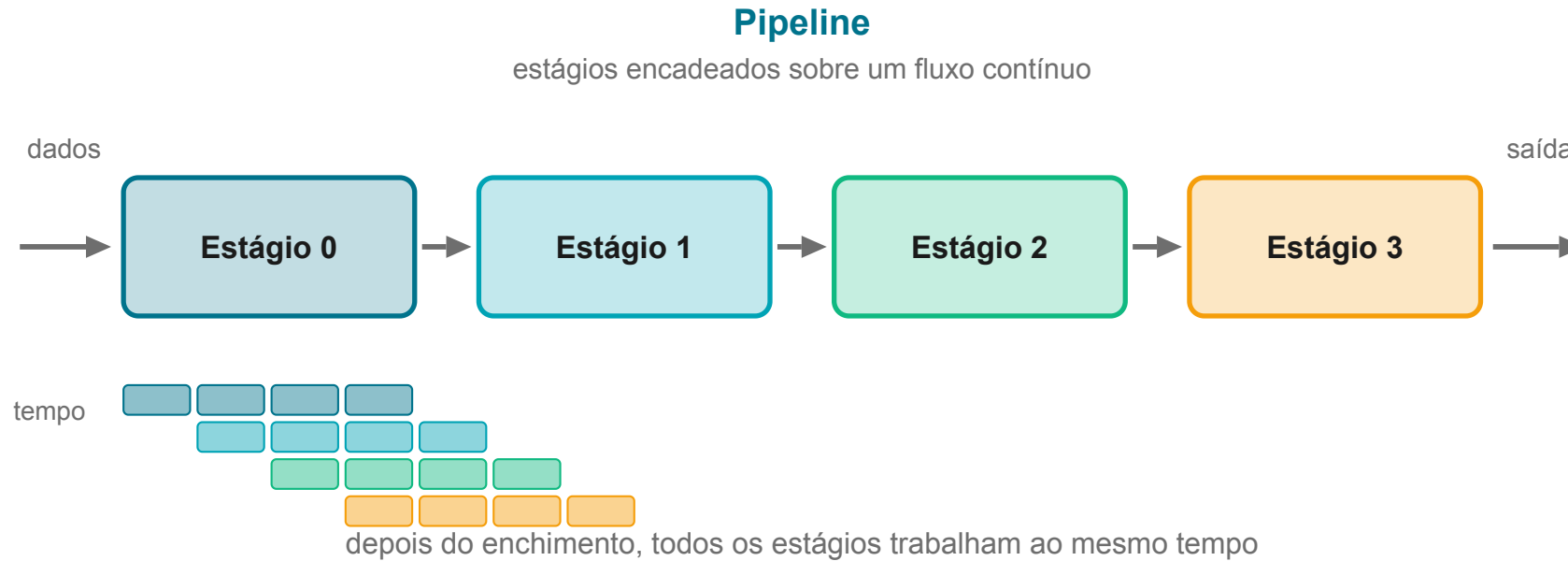
1. MESTRE / ESCRAVO



A favor: simples de implementar; tolera falha de escravo; balanceia naturalmente se as tarefas forem distribuídas sob demanda.

Contra: o mestre é gargalo de comunicação e ponto único de falha; escravos ociosos enquanto esperam.

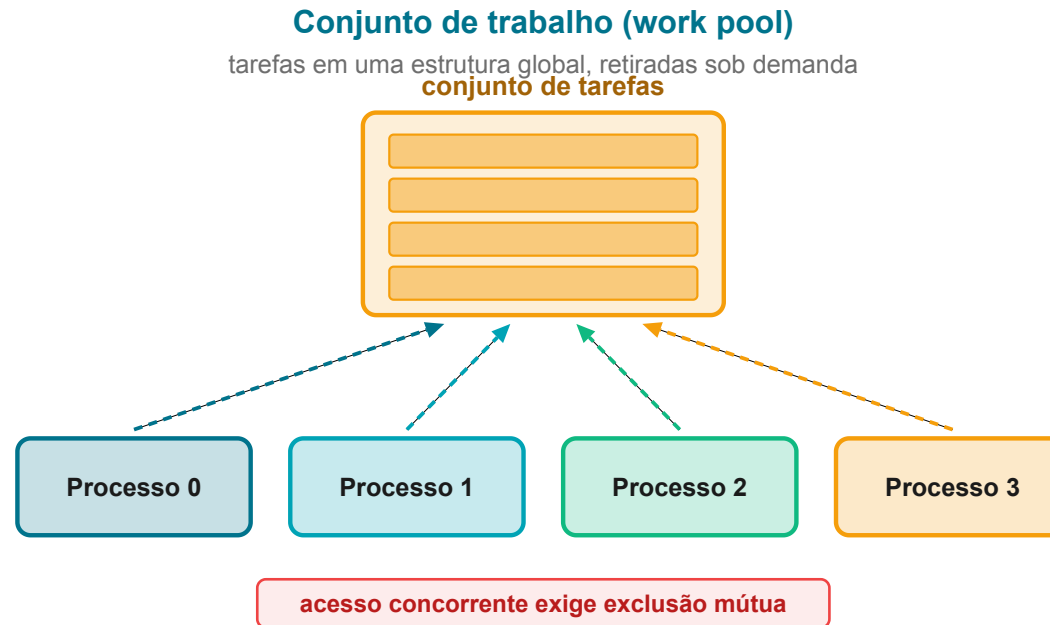
2. PIPELINE



A favor: sobrepõe comunicação e computação naturalmente; ideal para fluxo contínuo de dados.

Contra: não tolera falhas: um estágio para, todos param; o estágio mais lento limita a vazão; enchimento e esvaziamento desperdiçam tempo.

3. CONJUNTO DE TRABALHO



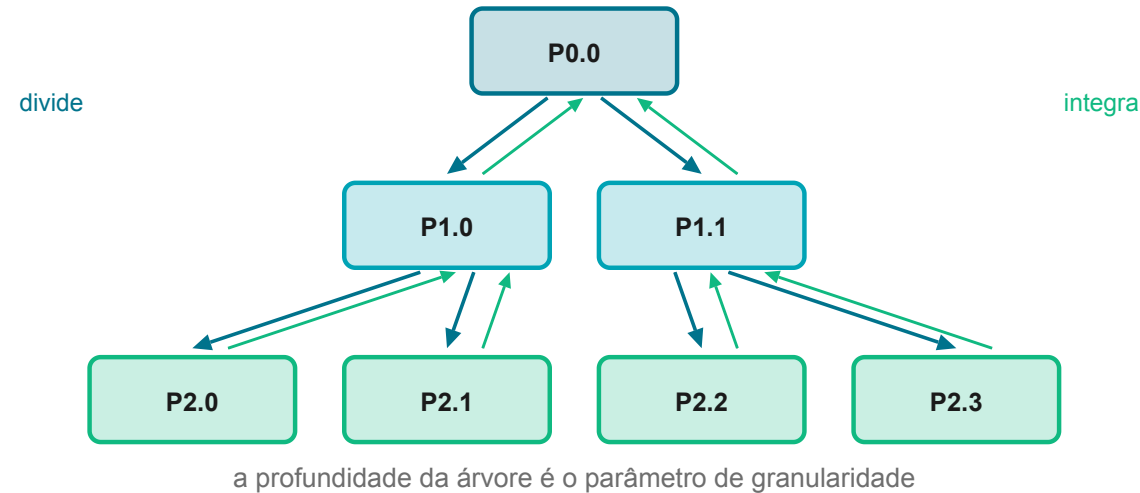
A favor: o melhor balanceamento de carga dos cinco; não exige estimar custo de tarefa; tarefas podem gerar novas tarefas.

Contra: exige sincronização no acesso à estrutura global; a contenção cresce com o número de processos.

4. DIVISÃO E CONQUISTA

Divisão e conquista

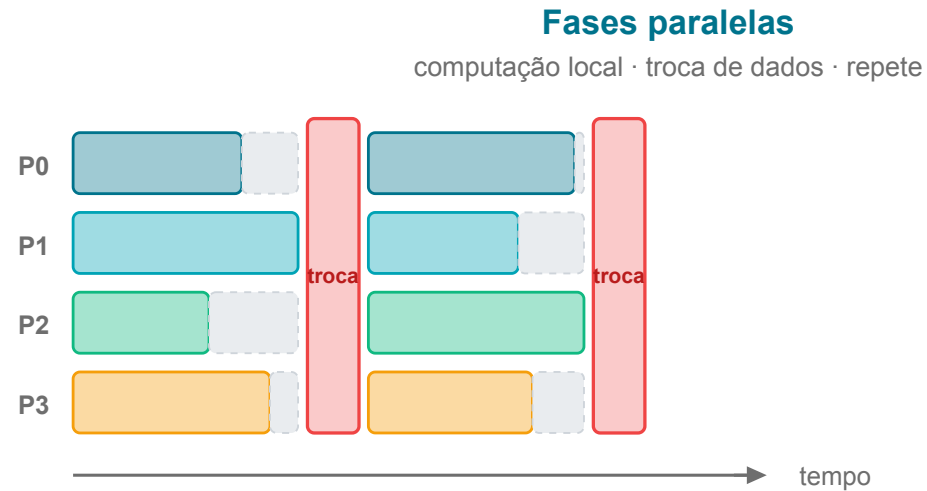
árvore de processos: divide na descida, integra na subida



A favor: mapeia diretamente algoritmos recursivos; a profundidade da árvore controla a granularidade.

Contra: difícil balancear se a divisão for desigual; nos níveis de cima quase todos os processos ficam ociosos.

5. FASES PARALELAS



blocos tracejados = espera na barreira

toda a comunicação acontece no mesmo instante — a rede satura em rajadas

A favor: estrutura simples e previsível; é o formato natural de simulações iterativas.

Contra: a barreira faz todos esperarem o mais lento; toda a comunicação ocorre no mesmo instante e a rede satura em rajadas.

OS CINCO LADO A LADO

Modelo	Granularidade	Balanceamento	Arquitetura	Quando usar
Mestre/Escravo	média a grossa	bom	cluster	tarefas independentes de custo variável
Pipeline	média	ruim	ambas	fluxo contínuo com etapas distintas
Conjunto de trabalho	fina a média	ótimo	compartilhada	custo imprevisível, tarefas geram tarefas
Divisão e conquista	variável	ruim	compartilhada	algoritmos recursivos
Fases paralelas	grossa	depende	cluster	simulações iterativas com vizinhança

Programas reais **combinam** modelos: fases paralelas por fora, divisão e conquista dentro de cada fase.

ESTUDO DE CASO

Um problema, cinco modelagens

COMO PARALELIZAR A ORDENAÇÃO DE UM VETOR GRANDE?

Um vetor com **10^9 elementos** (um bilhão). Vocês têm um cluster com 16 nós. Como fazem?

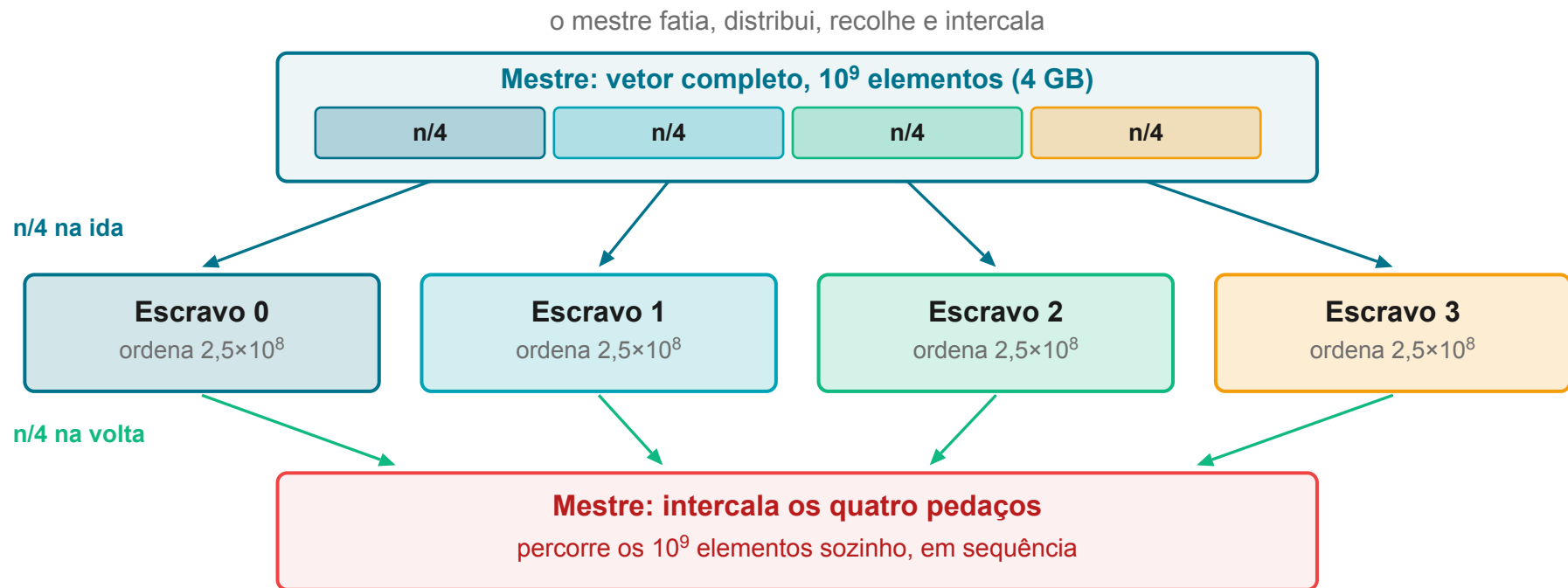
As perguntas que a modelagem precisa responder:

- Qual **paradigma de comunicação**?
- Distribuir partes do vetor ou **replicar**?
- Qual **modelo de aplicação**?

- Quantos **processos**?
- Quantos **processadores**?
- Onde **alocar** cada processo?

Vamos percorrer os cinco modelos e criticar cada um.

MODELAGEM 1: MESTRE / ESCRAVO



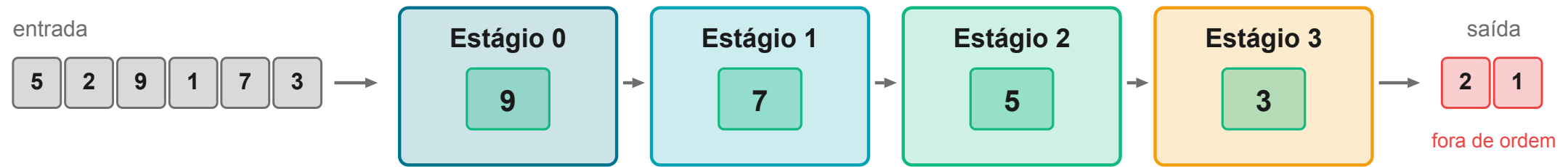
CRÍTICA

- O vetor passa pelo mestre **duas vezes**: ida e volta
- A intercalação final é sequencial, no mestre
- Com 10^9 elementos, o mestre nem armazena tudo

O custo de comunicação cresce com o tamanho do problema: não é um bom modelo aqui.

MODELAGEM 2: PIPELINE

bubble sort paralelo: uma comparação por estágio, sobre o fluxo



em verde, o maior valor que cada estágio já viu: o resto segue adiante

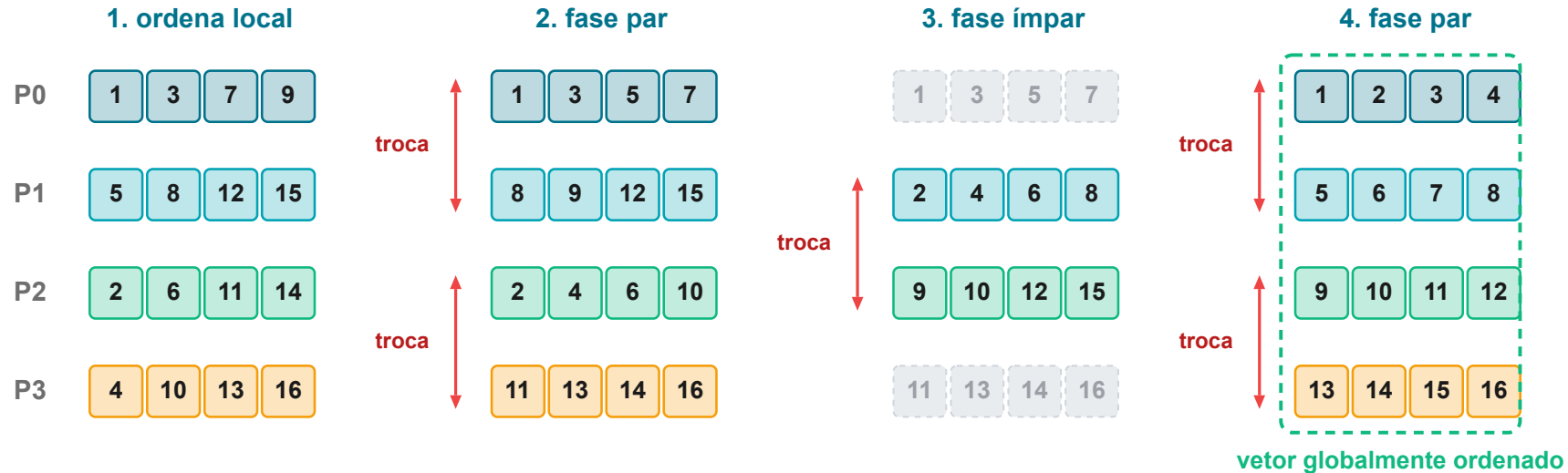
CRÍTICA

- Uma passagem posiciona apenas p elementos, um por estágio
- Com 16 nós, seriam 62 milhões de passagens
- Cada estágio faz trabalho trivial: grão fino demais

O número de estágios necessários cresce com n , mas o número de nós é fixo. Não escala.

MODELAGEM 3: FASES PARALELAS

odd-even transposition: ordena local, troca com o vizinho, repete



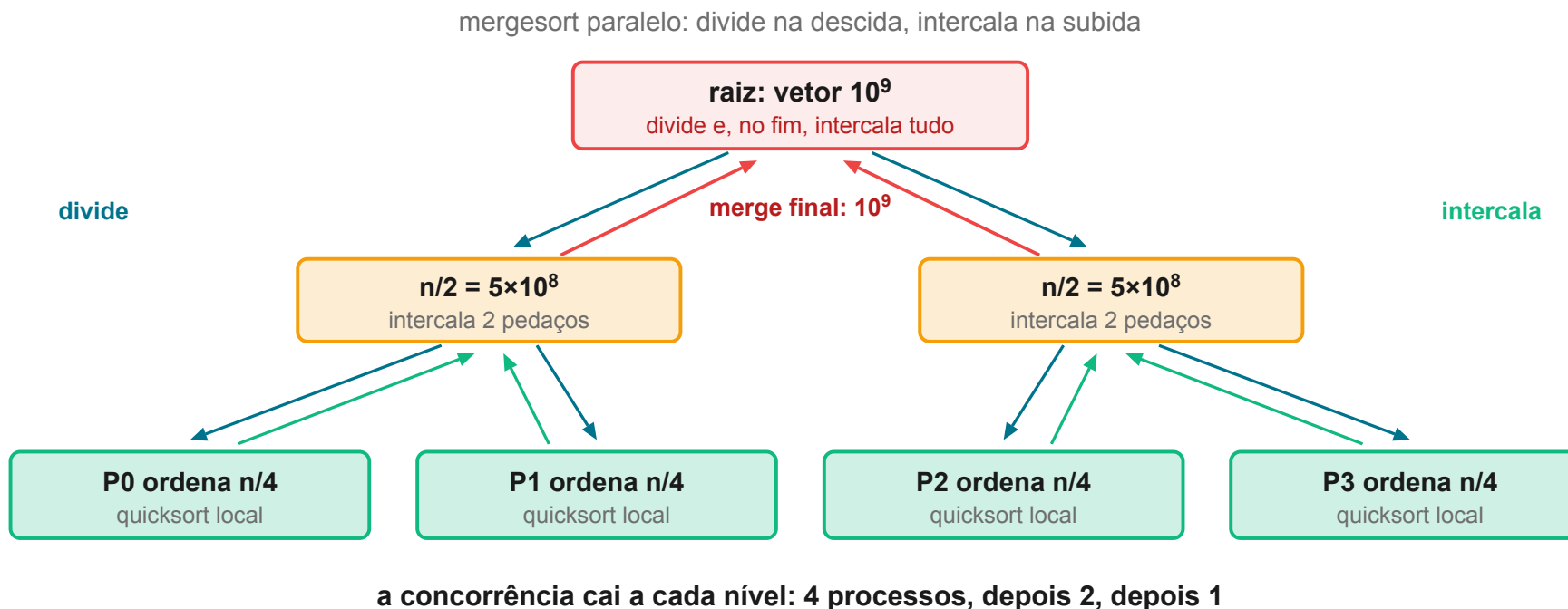
com p processos bastam p fases: nada é reunido em um único nó

CRÍTICA

- Cada processo guarda apenas n/p elementos
- Comunicação **local**: só com os vizinhos
- Grão grosso, e o volume trocado não cresce com n

Escala em memória e em comunicação. É a escolha natural para **troca de mensagens**.

MODELAGEM 4: DIVISÃO E CONQUISTA



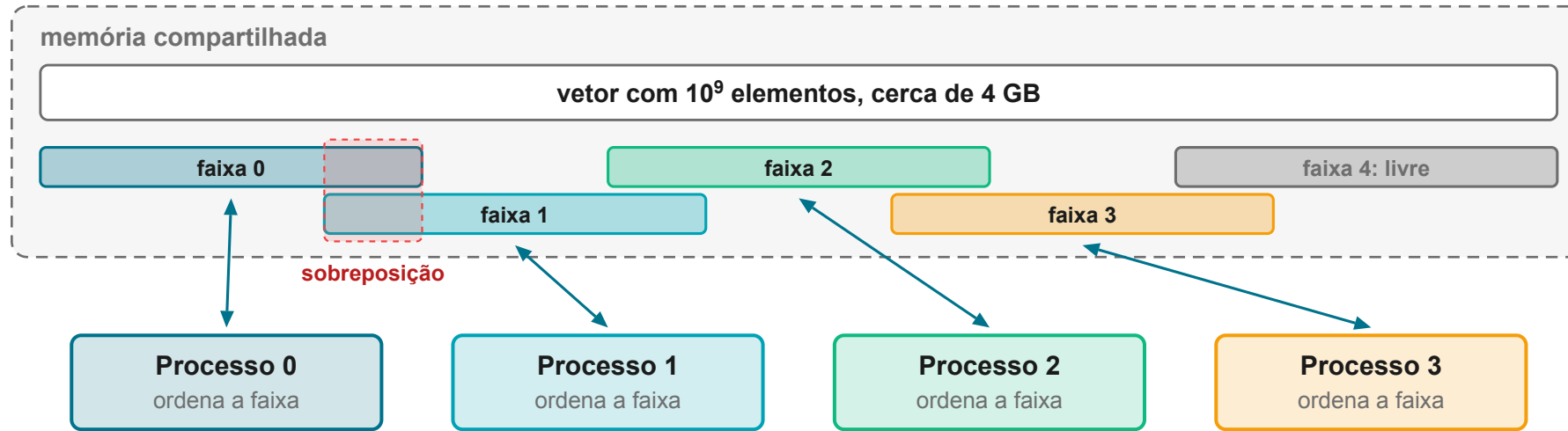
CRÍTICA

- A intercalação final processa o vetor **inteiro** em um nó
- Os níveis de cima têm pouca concorrência
- Em cluster, o merge final vira gargalo

Boa escolha dentro de um nó, com OpenMP. Problemática como estrutura global em cluster.

MODELAGEM 5: CONJUNTO DE TRABALHO

o vetor fica em memória compartilhada, dividido em faixas que se sobrepõem



retira uma faixa, ordena e devolve: quem termina antes pega a que sobrou

a sobreposição permite que um valor migre de uma faixa para a vizinha

CRÍTICA

- Exige o vetor **inteiro** em memória compartilhada
- Com 10⁹ elementos, não cabe em um nó só
- A contenção na estrutura global cresce com p

Balanceamento excelente, mas a memória compartilhada **elimina o cluster**.

VEREDITO

Modelo	Em cluster	Em multicore	Fator decisivo
Mestre/Escravo	ruim	ruim	Todo o vetor passa pelo mestre
Pipeline	ruim	ruim	Estágios necessários crescem com n
Conjunto de trabalho	inviável	razoável	Precisa do vetor em memória compartilhada
Divisão e conquista	limitado	bom	Intercalação final centralizada
Fases paralelas	bom	razoável	Comunicação local, memória distribuída

A resposta depende da máquina. Mesmo problema, mesmo algoritmo de base, modelagens diferentes conforme a arquitetura alvo.

Na prática, o híbrido: **fases paralelas** entre os nós, com **divisão e conquista** para a ordenação local dentro de cada nó.

MODELOS DE PROGRAMAÇÃO E INTERFACES

Da modelagem para a ferramenta

MODELO DE PROGRAMAÇÃO ≠ ARQUITETURA

São duas coisas independentes. Confundi-las é o mal-entendido mais comum vindo da Unidade 01.

Combinação	Possível?	Como
Compartilhada em multicore	sim	O caso natural: OpenMP, Pthreads
Mensagens em cluster	sim	O caso natural: MPI
Mensagens em multicore	sim	MPI em um só nó; a biblioteca usa memória, não rede
Compartilhada em cluster	sim	DSM e linguagens PGAS: mas o custo do acesso remoto não some

O modelo define **como** você escreve. A arquitetura define **quanto custa** o que você escreveu.

MODELO DE MEMÓRIA COMPARTILHADA

COMO FUNCIONA

- Espaço de endereçamento **único**
- Comunicação **implícita**: escrever numa variável
- Sincronização **explícita**: mutex, barreira, atômicas
- Paralelização **incremental**: dá para começar por um laço

O QUE DÁ ERRADO

- Condições de corrida silenciosas
- *Deadlock* entre travas
- *False sharing*
- Efeitos NUMA invisíveis no código

O perigo é justamente a facilidade: **esquecer de sincronizar compila e executa, e produz o resultado certo em 99 execuções de 100.**

MODELO DE TROCA DE MENSAGENS

COMO FUNCIONA

- Cada processo tem memória **privada**
- Comunicação **explícita**: `send` e `recv`
- Sincronização **embutida** na mensagem
- O particionamento dos dados é **obrigatório**

O QUE DÁ ERRADO

- *Deadlock* por ordenação de `send` / `recv`
- Custo de latência subestimado
- Mensagens não casadas
- Muito mais código para escrever

A obrigação de particionar explicitamente é **chata e saudável**: força o programador a enfrentar a modelagem em vez de adiá-la.

OS DOIS MODELOS LADO A LADO

Critério	Memória compartilhada	Troca de mensagens
Esforço para escrever	baixo	alto
Paralelização incremental	sim	não: é tudo ou nada
Escalabilidade	limitada ao nó	milhares de nós
Depuração	difícil : corridas silenciosas	erros mais visíveis, mas em escala
Controle da localidade	indireto	total e explícito

Não é uma escolha entre os dois. É uma escolha de **onde usar cada um**.

OPENMP

Interface para programação *multithread* em C, C++ e Fortran. Não é uma biblioteca: é um **padrão** implementado pelo compilador.

DIRETIVAS

```
#pragma omp ...
```

O compilador transforma em chamadas de criação de threads.

BIBLIOTECA

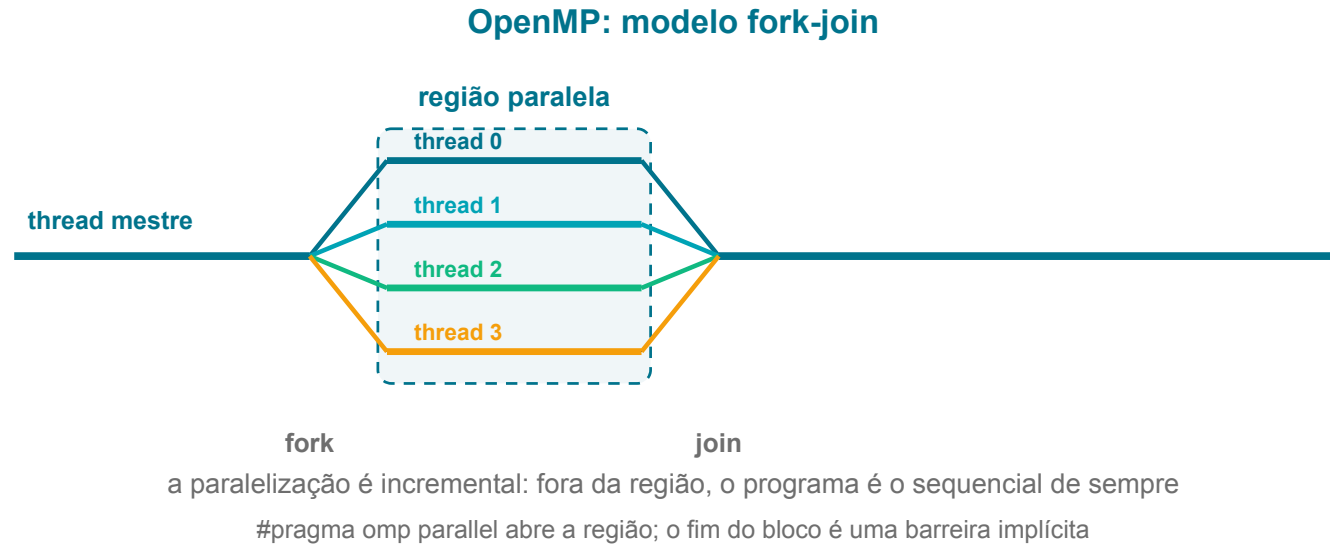
```
omp_get_thread_num(),  
omp_get_wtime(), travas.
```

VARIÁVEIS DE AMBIENTE

```
OMP_NUM_THREADS,  
OMP_SCHEDULE : mudam o  
comportamento sem recompilar.
```

Como as diretivas são *pragmas*, um compilador sem suporte a OpenMP simplesmente as ignora: e o programa continua compilando, sequencial.

OPENMP: FORK-JOIN



```
#include <omp.h>

int main() {
    printf("sequencial\n");
    #pragma omp parallel
    {
        printf("paralelo, thread %d\n", omp_get_thread_num());
    }
    printf("sequencial de novo\n");
}
```

OPENMP: A DIRETIVA QUE IMPORTA

```
int vet[100], soma = 0;

// paraleliza as iterações do laço entre as threads
#pragma omp parallel for
for (int i = 0; i < 100; i++)
    vet[i] = i * i;

// reduction: cada thread tem sua cópia de 'soma';
// no fim, o OpenMP combina todas com o operador +
#pragma omp parallel for reduction(+:soma)
for (int i = 0; i < 100; i++)
    soma += vet[i];
```

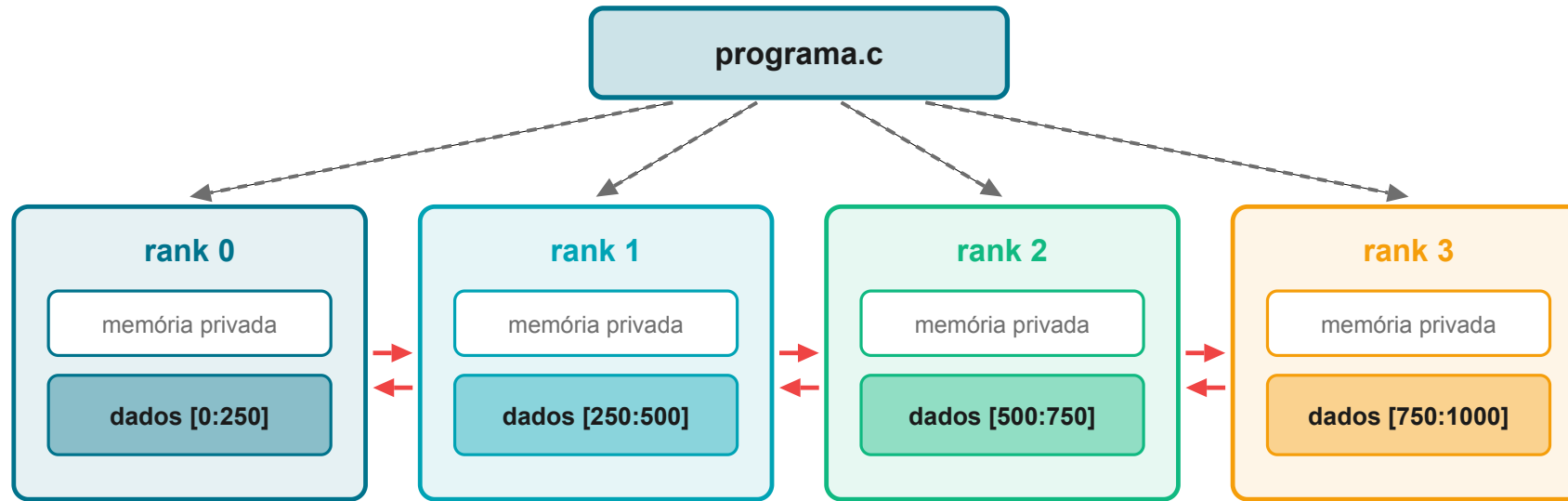
Sem `reduction`, o segundo laço tem **condição de corrida**: várias threads escrevendo em `soma` ao mesmo tempo. Compila, executa e dá resultado errado.

A cláusula `schedule(static | dynamic | guided)` é exatamente o que vimos no bloco de mapeamento.

MPI: MODELO SPMD

MPI: modelo SPMD

um único programa, vários processos, dados diferentes



não há nada compartilhado — toda troca é uma mensagem explícita

o rank é o que faz cada processo seguir um caminho diferente no mesmo código

MPI: AS OPERAÇÕES ESSENCIAIS

Categoria	Operações	Para quê
Controle	MPI_Init , MPI_Finalize , MPI_Comm_rank , MPI_Comm_size	Iniciar, encerrar, descobrir quem sou e quantos somos
Ponto a ponto	MPI_Send , MPI_Recv	Comunicação local entre vizinhos
Não-bloqueante	MPI_Isend , MPI_Irecv , MPI_Wait	Sobrepor comunicação e computação
Coletivas	MPI_Bcast , MPI_Reduce , MPI_Scatter , MPI_Gather	Comunicação global, implementada em árvore
Sincronização	MPI_Barrier	Alinhar todos os processos em um ponto

Cada linha desta tabela responde a uma decisão do bloco de **comunicação**: local ou global, bloqueante ou não, sobreposta ou não.

MPI: COMPILAR E EXECUTAR

```
# compilar: mpicc é um wrapper do gcc que já inclui os flags do MPI
mpicc programa.c -o programa
```

```
# executar com 4 processos na máquina local
mpirun -np 4 ./programa
```

```
# executar em várias máquinas, listadas no arquivo de hosts
mpirun --hostfile hosts.txt -np 16 ./programa
```

O QUE O `mpirun` FAZ

Dispara n cópias do mesmo executável, atribui um `rank` a cada uma e estabelece os canais de comunicação entre elas.

DETALHE IMPORTANTE

O número de processos é definido na **execução**, não na compilação. O mesmo binário roda com 4 ou 400.

O RESTO DO PANORAMA

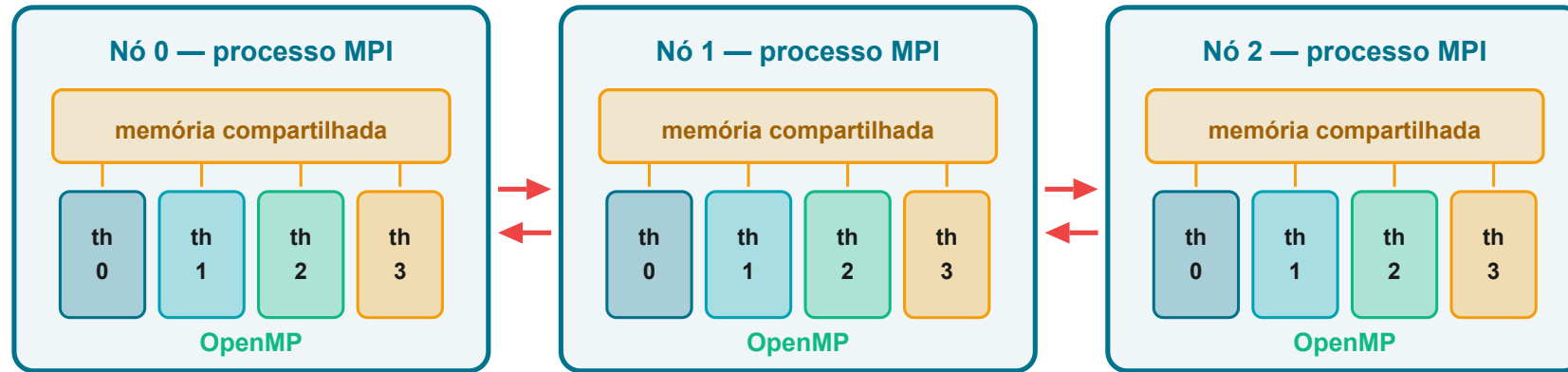
Categoria	Interfaces	Observação
Compartilhada	OpenMP, Pthreads, TBB, threads de linguagem (C++11, Java, Go, Rust)	Pthreads dá controle total e exige tudo à mão
Distribuída	MPI, PVM (histórico), Charm++	MPI é o padrão de fato desde os anos 1990
PGAS	UPC, Coarray Fortran, Chapel	Endereçamento global com noção de afinidade
Aceleradores	CUDA, OpenCL, OpenACC, <i>offloading</i> do OpenMP	Granularidade finíssima, milhares de threads
Alto nível	MapReduce, Spark, Dask	Abstraem a modelagem: e o controle sobre ela

Critérios de escolha: arquitetura alvo · granularidade viável · código legado · portabilidade · esforço *vs.* desempenho

O MODELO HÍBRIDO

Híbrido MPI + OpenMP

o modelo natural para um agregado de máquinas multicore



MPI entre os nós · OpenMP dentro do nó

evita pagar o custo de mensagens para comunicar dentro da mesma máquina

É o modelo natural para um **agregado de máquinas multicore**: exatamente o alvo declarado no plano de ensino desta disciplina.

O CAMINHO DAQUI EM DIANTE

UNIDADE 03

OpenMP

Paralelismo de laços, tarefas, sincronização, gerência de dados. Aqui você usa *aglomeração* e *mapeamento* o tempo todo.

UNIDADE 04

MPI

SPMD, bloqueante e não-bloqueante, coletivas, grupos. Aqui você usa *particionamento* e *comunicação*.

TRABALHO FINAL

Híbrido

Os dois juntos, com as métricas da Unidade 01 para provar que a modelagem funcionou.

A **P1** versa sobre as Unidades 01 e 02: arquiteturas, métricas e tudo o que vimos nesta unidade.

O MÉTODO, EM UMA TELA

Etapa	Pergunta	Verificação principal
Particionamento	Como quebrar em tarefas?	Há mais tarefas que processadores, e o número cresce com o problema?
Comunicação	Que dados atravessam fronteiras?	A comunicação é local, equilibrada e sobreponível?
Aglomeração	Quando parar de dividir?	O grão é maior que o custo de criá-lo e alimentá-lo?
Mapeamento	Quem executa o quê, e onde?	A carga está equilibrada e a localidade preservada?

E antes de tudo: **meça o programa sequencial** para descobrir onde vale a pena mexer.

As etapas não são um caminho de mão única. Descobrir um problema em C ou em M quase sempre significa **voltar e repartir de outro jeito.**

RESUMO

- Antes de decompor, **meça**: Amdahl diz que só vale mexer no que domina o tempo
- **PCAM**: particionar, comunicar, aglomerar, mapear; as duas primeiras sem olhar para a máquina
- A **granularidade ótima** é propriedade do par aplicação-máquina, nunca da aplicação sozinha
- **Balancear** e **preservar localidade** são objetivos que se contradizem
- Cinco **modelos de aplicação** cobrem quase todo programa paralelo: e podem ser combinados
- O **modelo de programação** define como se escreve; a **arquitetura**, quanto custa

A ferramenta é a última decisão do projeto, não a primeira.
Quem começa pela **diretiva** costuma terminar sem
escalabilidade.

"Qual é o caminho crítico do seu programa?"